



**COMSATS University Islamabad,
Sahiwal Campus.**

AgroAI: Smart Crop Disease Diagnosis System

A project presented to

**COMSATS University Islamabad,
Sahiwal Campus.**

In partial fulfillment

of the requirement for the degree of

Bachelor of Science in Computer Science (2022-2026)

By

Syed Rauf Haider Shah Gillani CIIT/FA22-BCS-117/SWL

Ali Shan Raza CIIT/FA22-BCS-202/SWL

DECLARATION

We hereby declare that this software, neither whole nor as a part has been copied out from any source. It is further declared that we have developed this software and accompanied report entirely on the basis of our personal efforts. If any part of this project is proved to be copied out from any source or found to be reproduction of some other. We will stand by the consequences. No Portion of the work presented has been submitted of any application for any other degree or qualification of this or any other university or institute of learning.

Syed Rauf Haider Shah Gillani

Ali Shan Raza

CERTIFICATE OF APPROVAL

It is to certify that the final year project of BS (CS) “**AgroAI: Smart Crop Disease Diagnosis System**” was developed by **SYED RAUF HAIDER SHAH GILLANI** (CIIT/FA22-BCS-117/SWL) and **ALI SHAN RAZA** (CIIT/FA22-BCS-202/SWL) under the supervision of “**Syed Nasir Mehdi**” and that in his opinion; it is fully adequate, in scope and quality for the degree of Bachelors of Science in Computer Sciences.

Supervisor

Syed Nasir Mehdi

Lecturer

Department of Computer Science

COMSATS University Islamabad, Sahiwal Campus

External Examiner

Dr. Ansar Munir Shah

Assistant Professor

Department of Computer Science

USP Multan

Head of Department

Dr. Zafar Iqbal Roy

Assistant Professor

Department of Computer Science

COMSATS University Islamabad, Sahiwal Campus

Executive Summary

Plant diseases are one of the most persistent and costly problems in agriculture. When a disease goes undetected for even a short period, it can spread across an entire crop, reducing yield quality and causing financial damage that is difficult to recover from. Despite how serious this problem is, most farmers and growers still rely on manual visual inspection or wait for access to an agricultural expert, both of which are slow, inconsistent, and not always available. Smallholder farmers are often left without a reliable way to get a fast, accurate diagnosis. AgroAI was developed to close this gap by providing a practical, AI powered platform that any user can access through a web browser or mobile device. AgroAI is developed to address this problem by providing an AI powered plant disease diagnosis platform that supports both web and mobile users. The system allows users to upload or capture plant leaf images and receive disease predictions through an integrated software environment.

AgroAI is implemented as a cross platform system so that users can interact with it conveniently through multiple environments. The mobile application is developed using Flutter, while the web application is implemented using the MERN stack with TypeScript. The backend is developed using Node.js and Express.js, MongoDB is used for storing user and diagnosis related information, and the plant disease detection model is maintained in .h5 format and deployed on Hugging Face Spaces for live inference. This structure enables the platform to combine frontend interaction, backend processing, persistent data storage, and AI based prediction within a single integrated system. The overall architecture follows a modular design in which the frontend provides user interaction, the backend coordinates requests and business logic, the database manages persistent records, and the AI inference layer performs disease classification.

The platform includes user registration and login, plant leaf image upload, disease prediction result display, diagnosis history, calendar based scan records, farm boundary map visualization, disease trends observation, and profile management. These features make AgroAI more than a simple prediction tool because it also provides organized record access and monitoring related support. Through requirement analysis, design, implementation, and testing, the project demonstrates the practical application of software engineering, artificial intelligence, database systems, web development, and mobile application development in the agricultural domain.

Acknowledgement

All praise is to Almighty Allah who bestowed upon us a minute portion of His boundless knowledge by virtue of which we were able to accomplish this challenging task.

We are greatly indebted to our project supervisor Syed Nasir Mehdi. Without their personal supervision, advice and valuable guidance, completion of this project would have been doubtful. We are deeply indebted to them for their encouragement and continual help during this work.

And we are also thankful to our parents and family who have been a constant source of encouragement for us and brought us the values of honesty & hard work.

Syed Rauf Haider Shah Gillani

Ali Shan Raza

Abbreviations

AI	Artificial Intelligence
ML	Machine Learning
DL	Deep Learning
MERN	MongoDB, Express.js, React, Node.js
HCI	Human Computer Interaction
API	Application Programming Interface
UC	Use Case
FR	Functional Requirement
NFR	Non Functional Requirement
ID	Identifier
FYP	Final Year Project
UI	User Interface

Table of Contents

1	Introduction.....	1
1.1	Brief	2
1.2	Relevance to Course Modules	2
1.3	Project Background.....	6
1.4	Literature Review.....	8
1.5	Analysis from Literature Review	12
1.6	Methodology and Software Lifecycle.....	14
1.6.1	Rationale behind Selected Methodology	15
2	Problem Definition.....	17
2.1	Problem Statement	17
2.2	Deliverable and Development Requirements	18
2.3	Current System.....	19
3	Requirement Analysis	21
3.1	Use Case Diagram.....	21
3.2	Detailed Use Cases	23
3.2.1	Use Case: Register Account.....	23
3.2.2	Use Case: Login to System	24
3.2.3	Use Case: Upload Leaf Image	25
3.2.4	Use Case: View Diagnosis Result.....	26
3.2.5	Use Case: View Diagnosis History	27
3.2.6	Use Case: View Disease Trends.....	27
3.2.7	Use Case: View Farm on Map	28
3.2.8	Use Case: View Scan Dates in Calendar.....	29
3.2.9	Use Case: Manage User Profile	30
3.2.10	Use Case: Logout from System	31
3.3	Functional Requirements	31
3.3.1	Description of Functional Requirements	32
3.4	Non Functional Requirements	33
3.4.1	Description of Non Functional Requirements	33
4	Design and Architecture.....	34
4.1	System Architecture	34

4.2	Data Representation	38
4.3	Process Flow / Representation	42
4.4	Design Models	44
4.4.1	Sequence Diagram	44
4.4.2	Class Diagram.....	45
5	Implementation	47
5.1	Algorithm.....	47
5.1.1	AgroNet CNN Architecture and Working.....	49
5.1.2	Image Upload and Validation Process	51
5.1.3	AI Model Inference Workflow.....	53
5.1.4	Backend Coordination Logic	54
5.1.5	Authentication Algorithm	55
5.1.6	Diagnosis Record Storage Algorithm	56
5.1.7	Diagnosis History Retrieval Algorithm	57
5.1.8	Disease Trends and Calendar Data Logic	58
5.1.9	Error Handling in Algorithmic Workflow.....	59
5.2	External APIs	60
5.3	User Interface.....	62
6	Testing and Evaluation.....	68
6.1	Manual Testing.....	68
6.1.1	System Testing	70
6.1.2	Unit Testing.....	72
6.1.3	Functional Testing.....	73
6.1.4	Integration Testing	75
6.2	Automated Testing	76
6.3	AI Model Training Results and Evaluation	77
7	Conclusion and Future Work	79
7.1	Conclusion	79
7.2	Future Work	81
	References.....	84

List of Tables

Table 3.2.1 Use Case Register Account	24
Table 3.2.2 Use Case Login to System	24
Table 3.2.3 Use Case Upload Leaf Image.....	25
Table 3.2.4 Use Case View Diagnosis Result	26
Table 3.2.5 Use Case View Diagnosis History	27
Table 3.2.6 Use Case View Disease Trends	28
Table 3.2.7 Use Case View Farm on Map.....	28
Table 3.2.8 Use Case View Scan Dates in Calendar	29
Table 3.2.9 Use Case Manage User Profile	30
Table 3.2.10 Use Case Logout from System.....	31
Table 3.3.1 Functional Requirements of AgroAI.....	32
Table 3.4.1 Non Functional Requirements of AgroAI.....	33
Table 4.2.1 Data Representation of AgroAI.....	41
Table 5.1.1 AgroNet Layer-by-Layer Architecture	49
Table 5.1.2 Diagnosis Record Storage Algorithm	56
Table 5.1.3 Disease Trends and Calender Data Logic	59
Table 5.1.4 Error Handling in Algorithmic Workflow	60
Table 5.2.1 External APIs of AgroAI.....	62
Table 6.1.1 System Testing of AgroAI.....	71
Table 6.1.2 Unit Testing of AgroAI	73
Table 6.1.3 Functional Testing of AgroAI	74
Table 6.1.4 Integration Testing of AgroAI.....	76
Table 6.2.1 Automated Testing of AgroAI.....	77
Table 6.3.1 AgroNet Training Configuration	78

List of Figures

Figure 3-1 Use Case Diagram of AgroAI	23
Figure 4-1 System Architecture of AgroAI.....	38
Figure 4-2 Data Representation of AgroAI.....	40
Figure 4-3 Flow Representation of AgroAI	44
Figure 4-4 Sequence Diagram of AgroAI.....	45
Figure 4-5 Class Diagram of AgroAI.....	46
Figure 5-1 Homepage of AgroAI.....	65
Figure 5-2 Registration of AgroAI.....	65
Figure 5-3 Sign In page of AgroAI.....	66
Figure 5-4 Dashboard of AgroAI.....	66
Figure 5-5 Scan page of AgroAI.....	67
Figure 5-6 Crop Calendar of AgroAI.....	67
Figure 5-7 Disease Trends of AgroAI.....	68
Figure 5-8 Profile Page of AgroAI.....	68
Figure 6-1 AgroNet Training and Validation Accuracy	78

1 Introduction

AgroAI is developed as a smart crop health support system that uses artificial intelligence to assist in plant disease identification. The project focuses on a common agricultural problem: users often notice visible changes on plant leaves but may not know whether those changes are caused by disease, environmental stress, or normal plant aging. In such situations, a quick and accessible digital tool can help users make an initial diagnosis and take timely action.

The main idea behind AgroAI is to provide a simple platform where a user can submit a plant leaf image and receive an AI based prediction. Instead of depending only on manual observation, the system uses a trained image classification model to analyze visual patterns from the uploaded leaf image. This makes the diagnosis process more structured and reduces uncertainty for users who do not have expert agricultural knowledge.

The project is designed as a cross platform application so that it can be used in both web and mobile environments. The web application is built using the MERN stack with TypeScript, while the mobile application is developed using Flutter. The backend handles request processing, image submission, authentication, and communication with the AI service. MongoDB is used to store user data, diagnosis records, and other system related information. The AI model is stored in .h5 format and deployed through Hugging Face Spaces, allowing the system to perform live disease prediction.

AgroAI is not limited to only displaying a prediction result. It also provides supporting features such as user registration, login, diagnosis history, disease trend viewing, calendar based scan records, saved farm boundary visualization, and profile management. These features make the system more useful because users can review their previous scans, observe disease activity over time, and manage their crop related information in a more organized way.

This project represents the practical application of computer science in the agriculture domain. It combines artificial intelligence, full stack web development, mobile application development, database management, and software engineering principles into one working system. Through AgroAI, plant disease diagnosis becomes more accessible, faster, and easier to manage, especially for users who need a convenient digital method for crop health monitoring.

1.1 Brief

AgroAI is an AI based plant disease diagnosis system that was created to offer an accessible, practical, and modern software solution for analyzing plant health. The project aims to help users detect plant diseases through image based prediction, where plant leaf images are uploaded or captured and then analyzed by an artificial intelligence model. The purpose of the system is to reduce complete dependence on manual diagnosis, which is often slow, inconsistent, and difficult for users who do not have expert agricultural knowledge. In traditional agriculture, plant diseases are mostly identified through visual inspection or specialist advice. These methods are not always timely, accurate, or conveniently available. AgroAI addresses this issue by providing a digital platform where plant leaf images can be analyzed using artificial intelligence.

The system is designed as a cross platform solution so that it can be accessed through both web and mobile environments. This makes the project more useful because users can interact with it from different devices according to their convenience. When a user uploads a plant leaf image, the image is sent from the frontend to the backend, where it is validated and forwarded to the AI model for analysis. After processing the image, the model returns the predicted disease result along with confidence related information. The backend then manages the response, stores the diagnosis record in MongoDB, and sends the result back to the user interface for display.

AgroAI also supports important application features such as user registration, login, plant image upload, diagnosis result display, diagnosis history, disease trends, scan calendar, saved farm boundary map, profile management, and logout. These features make the system more than a basic prediction tool. It allows users to maintain their diagnosis records, review previous scans, observe disease related patterns, and interact with crop health information in a more organized way. The inclusion of history, trends, calendar, and map related features increases the practical value of the platform and makes it more suitable for real world use.

1.2 Relevance to Course Modules

The knowledge and skills acquired throughout our Bachelor of Science in Computer Science program have played an important role in the successful planning, development, and execution of AgroAI. This project is not limited to one specific course or technical area; instead, it

combines multiple academic concepts into a single practical software system. Each course contributed to a different part of the project, either by providing theoretical knowledge or by supporting practical implementation. AgroAI required understanding of software development, database management, artificial intelligence, web technologies, mobile application development, user interface design, and project management. Therefore, the project reflects the interdisciplinary nature of computer science and shows how classroom learning can be transformed into a real world application.

i. Software Engineering Concepts

Software Engineering is one of the most relevant course modules for AgroAI because the project follows a structured software development process from requirement gathering to final testing. The concepts learned in Software Engineering helped in applying important ideas such as the Software Development Life Cycle (SDLC) and requirement gathering. These concepts supported the identification of user needs, preparation of use cases, design of system architecture, implementation of modules, and testing of the completed system.

AgroAI includes several important features such as user registration, login, image upload, disease prediction, diagnosis history, calendar view, disease trends, farm map, and profile management. These features were not developed randomly they were planned according to the gathered requirements and organized into functional parts of the system. Software Engineering concepts also helped in preparing proper documentation, diagrams, requirement analysis, and testing plans.

ii. Artificial Intelligence and Machine Learning

Artificial Intelligence and Machine Learning are central to AgroAI because the system uses an AI based plant disease detection model to analyze plant leaf images. The knowledge gained from AI and Machine Learning courses helped in understanding important concepts such as supervised learning, image classification, deep learning, model training, prediction, inference, confidence score, and pattern recognition. These concepts are directly applied in AgroAI to identify possible plant diseases from uploaded leaf images.

When a user uploads a plant leaf image, the model analyzes the visual patterns present in the image and returns the predicted disease result with confidence related information. This practical use of AI and ML shows how trained models can be used to solve a real agricultural problem by supporting faster, more accessible, and more structured plant disease diagnosis.

iii. Database Systems

Database Systems played an important role in the development of AgroAI because the system requires proper storage, retrieval, and management of user related and diagnosis related data. The concepts learned in Database Systems helped in understanding important ideas such as data modeling, database design, record management, data retrieval, user specific data access, and data consistency.

MongoDB is used to store user accounts, uploaded image references, diagnosis results, scan history, profile information, and other related system data. Since each user has personal diagnosis records, the database must keep every record correctly linked to the authenticated user. This ensures that users can only access their own history, profile, scan calendar, and disease related records. Therefore, database design is an essential part of AgroAI because it supports organized data storage and reliable access to system information.

iv. Web Development

AgroAI is strongly related to Web Engineering because the web application is developed using the MERN stack with TypeScript. React with TypeScript is used for creating the frontend interface, while Node.js and Express.js are used for backend services. The web engineering concepts applied in the project include client server communication, routing, API handling, form validation, request response processing, and integration between frontend and backend. Through the web application, users can register, log in, upload plant leaf images, view diagnosis results, access history, and manage their profiles from a browser. This part of the project shows how web technologies can be used to build an interactive and functional platform.

v. Mobile Application Development

Mobile Application Development is another important course module connected to AgroAI because the system also includes a mobile application developed using Flutter. The mobile application provides users with a convenient way to access the plant disease diagnosis system from their smartphones. This is especially useful because crop related activities often take place outside desktop environments, and users may need to capture or upload plant images directly from mobile devices. The concepts of mobile screen design, navigation, image selection, responsive layouts, and API integration are applied in this part of the system. Flutter supports

cross platform mobile development, which makes it suitable for developing a smooth and accessible mobile interface.

vi. Human Computer Interaction

Human Computer Interaction is relevant to AgroAI because the success of the system depends not only on its technical functionality but also on how easily users can operate it. The concepts learned in HCI helped in understanding important ideas such as usability, accessibility, user friendly interface design, consistency, feedback, navigation, and error prevention.

AgroAI is designed for users who may not have advanced technical knowledge, so the interface must be simple, clear, and easy to understand. HCI concepts helped in designing screens such as the dashboard, image upload page, diagnosis result page, history page, calendar, disease trends, farm map, and profile section. A good interface reduces confusion and allows users to complete important tasks quickly, such as uploading a leaf image, viewing the diagnosis result, and checking previous scan records. In AgroAI, usability is important because users should be able to interact with the system without facing unnecessary complexity.

vii. Software Project Management

Software Project Management is connected to AgroAI because the project required proper planning, scheduling, task division, technology selection, testing, and documentation. The concepts learned in this course helped in managing the project in an organized way and ensuring that each module was completed step by step.

AgroAI uses Agile methodology with an iterative lifecycle, which supports gradual development and continuous improvement. Since the project includes different modules such as the web application, mobile application, backend, database, and AI inference layer, proper project management was necessary to keep the development process controlled and coordinated.

Agile methodology helped in dividing the project into smaller development stages. Each stage focused on specific features such as authentication, image upload, disease prediction, diagnosis history, calendar view, disease trends, and profile management. This approach made the project easier to manage, allowed testing after each feature, and reduced the risk of major problems appearing at the final stage.

viii. System Analysis and Design

System Analysis and Design concepts are applied in AgroAI through use case diagrams, sequence diagrams, class diagrams, architecture diagrams, data representation, and process flow representation. These diagrams help explain how the system works and how different components interact with each other. They also provide a clear structure before implementation begins. For example, the process flow explains how the user uploads an image, how the backend sends it to the AI model, how the prediction is returned, and how the result is stored and displayed. These design models make the system easier to understand and improve the quality of documentation.

Overall, the multidisciplinary nature of the Computer Science degree program helped in developing AgroAI as a complete and practical final year project. The project combines software engineering practices, AI based prediction, database storage, web and mobile development, user interface design, and project management into one integrated platform. Each course module contributed to a specific part of the system, and the combination of these modules made it possible to transform AgroAI from an idea into a functional crop disease diagnosis system.

1.3 Project Background

Agriculture is one of the most important sectors in many countries because it directly supports food production, economic activity, employment, and livelihood. A large portion of the population depends on agriculture either directly or indirectly, which makes crop health an important factor for both individual farmers and the overall agricultural economy. Healthy crops result in better yield, improved quality, and stable production, while unhealthy crops can create serious financial and food related problems. Among the different challenges faced in agriculture, plant disease is one of the most common and damaging issues because it can reduce crop productivity within a short period if it is not identified and controlled on time.

Plant diseases usually appear through visible symptoms such as spots, discoloration, dryness, fungal marks, or unusual patterns on leaves. These symptoms are not always easy to understand. Diseases often show similar signs, and a person without expert agricultural knowledge may not be able to identify the exact problem correctly. In traditional farming practices, disease identification is mostly done through manual observation or by consulting

agricultural experts. Although these methods can be helpful, they are often slow, uncertain, and not always available when needed. In some areas, farmers may not have quick access to specialists, and delay in diagnosis can allow the disease to spread across the crop. This can lead to low yield, poor crop quality, and financial loss.

The background idea of AgroAI was developed from this practical agricultural problem. The project was planned to provide a software based platform that can assist users in plant disease diagnosis through image based prediction. Instead of depending only on manual inspection, the user can upload or capture an image of a plant leaf and receive a disease prediction generated through artificial intelligence. This makes the diagnosis process more organized, accessible, and faster. AgroAI does not aim to completely replace agricultural experts; rather, it provides an initial digital support system that can help users recognize possible crop diseases and take timely action.

The rise of artificial intelligence, especially image classification and deep learning, has created new opportunities in the agricultural domain. Modern AI models can be trained to recognize visual patterns in plant leaf images and classify them into healthy or diseased categories. This ability makes artificial intelligence suitable for plant disease detection because many crop diseases are visually reflected on leaves. By using trained models, software systems can analyze an uploaded image and produce a prediction based on learned patterns. This provides a practical way to bring intelligent decision support into agriculture through digital platforms.

AgroAI is based on the idea of combining artificial intelligence with a complete web and mobile software system. The project is not only focused on creating a disease prediction model; it also focuses on making that model usable through a proper application interface. The system allows users to interact with the AI model through familiar platforms such as a mobile application and a web application. This is important because a trained model alone has limited practical value unless it is connected with a system where users can easily upload images, view results, and store diagnosis records.

In its real world implementation, AgroAI uses Flutter for the mobile application and the MERN stack with TypeScript for the web application. React with TypeScript is used to build the web frontend, while Node.js and Express.js are used for backend services. MongoDB is used for storing user data, diagnosis records, scan history, and other system related information. The plant disease detection model is stored in .h5 format and deployed on Hugging Face Spaces to

support live prediction. This technical combination allows the system to connect frontend interaction, backend processing, database storage, and AI inference in a structured way.

The motivation behind AgroAI comes from the observation that the technologies required for a useful plant disease diagnosis system already exist, but they need to be combined into a complete and accessible platform. Trained deep learning models can support image classification, cloud based deployment platforms can provide live inference, and cross platform development frameworks can make the system available to more users. AgroAI brings these elements together into one system so that users can perform disease diagnosis through a simple digital process. The project therefore applies proven computer science techniques to a real agricultural problem in a practical and user oriented manner.

Overall, the project background of AgroAI is rooted in the need for timely, accessible, and technology supported plant disease detection. By combining artificial intelligence, mobile application development, web technologies, backend services, and database management, AgroAI provides a modern approach to crop health diagnosis. It represents the use of computer science for solving an agriculture related issue and shows how intelligent software systems can support users in making faster and more informed decisions about plant health.

1.4 Literature Review

Over the last several years, much work has been conducted in the field of plant disease detection, smart agriculture, and AI enabled crop monitoring. Current systems and literature demonstrate that technology has strong potential to improve the speed, accuracy, and quality of agricultural decision making, especially in disease diagnosis. Plant disease identification has traditionally depended on human observation, but modern research shows that image processing, machine learning, and deep learning can support this process in a more systematic way. A wide range of contemporary solutions use plant leaf images as input and apply artificial intelligence techniques to recognize visual symptoms such as spots, color changes, lesions, and other disease patterns. These developments show that agriculture is moving from manual and experience based diagnosis toward data driven and software supported diagnosis.

One of the most important areas discussed in the literature is image based plant disease detection. In this approach, the system analyzes images of leaves or other plant parts and attempts to classify the condition of the plant. Earlier methods used traditional image

processing techniques, such as color segmentation, texture analysis, and feature extraction. These methods required manually selected features and often depended heavily on image quality and background conditions. With the rise of deep learning, especially convolutional neural networks, plant disease detection became more effective because the model can automatically learn useful visual features from training images. This made deep learning more suitable for disease classification tasks where symptoms vary in shape, size, color, and intensity.

Several existing systems and applications have shown how artificial intelligence can be used in practical agricultural environments. **Plantix** is an example, a mobile based plant disease detection application that allows users to capture or upload plant images and receive disease related information and crop care suggestions. Plantix shows that image based plant disease detection can be made available to real users through mobile technology. It also demonstrates that farmers and growers can benefit from a system that provides quick feedback instead of waiting for manual expert inspection. Such systems usually follow their own fixed usage model and may not always provide full flexibility for academic customization, cross platform expansion, or integration with separate backend and database components.

Leaf Doctor is another example, which is often discussed as a plant disease analysis tool focused on measuring leaf disease severity. It is useful from a research and analysis perspective because it supports disease measurement and visual symptom evaluation. Leaf Doctor is not mainly designed as a complete cross platform user system for ordinary users. Its focus is more limited compared with a full software platform that includes user accounts, history, dashboard features, mobile access, web access, and database supported record management. This shows that some systems are valuable for research purposes but may not fully address the need for a complete and accessible diagnosis platform.

PlantVillage has contributed significantly to plant disease detection research by providing a large image dataset of healthy and diseased plant leaves. PlantVillage images are used by students to train and test deep learning models for disease classification. The importance of PlantVillage is that it has supported the development of many AI based plant disease detection models by offering organized image data. A dataset alone is not a complete user facing solution. It helps in model development and experimentation, but it still requires a proper software system to make disease diagnosis available to actual users. AgroAI is related to this direction because it focuses on connecting a trained model with a working web and mobile application.

Similarly, research projects based on convolutional neural networks have shown strong results in plant disease classification. CNN based models are widely used because they are able to learn patterns directly from images. These models can identify differences between healthy leaves and diseased leaves by analyzing visual features. For example, symptoms such as brown spots, yellow patches, fungal marks, leaf blight patterns, and irregular discoloration can be recognized by a trained model. This makes CNNs very suitable for agricultural image classification. Most of these research models remain limited to experiments, datasets, or offline testing environments. They may achieve good accuracy in controlled conditions but are not always connected to a full application that ordinary users can easily access.

There are also smart agriculture systems that focus on crop monitoring through sensors, IoT devices, drones, or remote sensing. These systems collect agricultural data such as soil moisture, temperature, humidity, crop growth status, and aerial crop images. Drone based crop monitoring systems, for example, can observe large fields and identify areas affected by disease or stress. Remote sensing systems can also help in detecting crop health issues at a wider scale. These technologies are useful for large farms and advanced agricultural management, but they may require expensive hardware, technical skills, or specialized infrastructure. In contrast, an image based software system like AgroAI is more accessible because it can work with a simple image upload from a web or mobile application.

Mobile based agriculture applications have also become more common in recent years. These applications provide services such as weather updates, crop advisory, pest control suggestions, fertilizer guidance, market prices, and disease detection. The popularity of these applications shows that farmers and agriculture related users are increasingly willing to use digital tools for decision making. Most applications focus only on advisory content and do not include AI based image diagnosis. Others may include disease detection but lack strong record management, trend analysis, or cross platform support. This creates a gap for systems that combine AI prediction with proper software features such as user authentication, diagnosis history, calendar records, disease trends, and map based farm visualization.

The literature also highlights the importance of user friendliness in agricultural technology. A system may have a strong AI model, but if the interface is difficult to use, its practical value becomes limited. Farmers and general users need simple steps: upload an image, submit it for analysis, view the result, and access previous records when needed. Therefore, the success of a plant disease diagnosis platform depends not only on model accuracy but also on usability,

accessibility, and clear presentation of results. This is why many modern agricultural systems are moving toward mobile and web interfaces rather than remaining as desktop only tools or lab based prototypes.

Existing work also highlights the need for organized record keeping. Plant disease diagnosis is not always a one time activity. Users may want to compare previous scans, observe repeated disease patterns, and track crop health activity over time. Simple AI prediction tools often provides a result only for the current image and do not focus on long term record management. AgroAI addresses this need by including features such as diagnosis history, disease trends, scan calendar, and farm boundary map visualization. These features make the system more useful because they allow users to review and monitor crop related information in a structured way.

The review of existing systems and research confirm that AI based plant disease detection is no longer just an experimental idea. It has been demonstrated successfully in controlled environments and practical applications. Deep learning models, especially convolutional neural networks, have become a common approach for identifying plant diseases from images. Platforms such as Plantix show the practical value of mobile based disease diagnosis, while tools such as Leaf Doctor and datasets such as PlantVillage show the research importance of image based disease analysis. Drone based and IoT based smart agriculture systems also show that digital agriculture is expanding rapidly.

The literature also shows that there is still a need for systems that are accessible, integrated, and usable across different platforms. Existing solutions are limited to mobile only use, research only tools, offline models, or hardware dependent systems. Some focus mainly on prediction but do not provide proper user history, trend viewing, calendar records, or farm map support. AgroAI is developed within this gap. It does not aim to introduce a new AI algorithm or a completely new model architecture. Instead, it focuses on applying existing AI based plant disease detection methods in a complete software system that can be used through web and mobile platforms.

Overall, the reviewed literature and related systems support the need for AgroAI. They show that artificial intelligence can successfully identify plant disease patterns from images, but they also show that practical delivery remains an important challenge. AgroAI contributes by combining a trained .h5 plant disease detection model, Hugging Face Spaces deployment, Flutter mobile application, MERN stack web application, MongoDB database, and user centered features into one integrated system. In this way, AgroAI follows the direction of

modern smart agriculture while focusing on accessibility, usability, and practical software implementation.

1.5 Analysis from Literature Review

The analysis of the reviewed systems and related project material shows that AgroAI shares the same overall objective as many existing plant disease detection platforms, which is to improve disease identification through digital and intelligent methods. Existing applications and research based systems show that artificial intelligence can be used to identify plant diseases from images and support agricultural decision making. The main difference lies in how these systems are delivered to users and how completely they are integrated into a usable software platform. Many systems are either limited to research demonstrations, single purpose tools, mobile only applications, or offline model experiments. AgroAI differs from these approaches by focusing on practical software delivery along with AI based prediction.

One major point identified from the literature is that image based plant disease detection has already become a reliable direction in smart agriculture. Deep learning models, especially convolutional neural networks, have shown strong performance in recognizing disease symptoms from plant leaf images. These models can detect patterns such as spots, color changes, fungal marks, and damaged leaf areas. This confirms that the use of AI for disease diagnosis is suitable for the AgroAI project. Research models alone are not enough for practical use. A model may perform well in testing, but it becomes useful to general users only when it is integrated into a complete system with a proper interface, backend processing, database storage, and result display.

The review further finds that existing platforms such as Plantix demonstrate the practical value of mobile based plant disease detection. Such platforms show that users can benefit from quick image based diagnosis and crop related guidance. Most existing platforms operate within fixed usage models and may not provide flexibility for academic design, customization, or full system control. Similarly, research based tools such as Leaf Doctor are useful for studying disease severity and visual symptoms, but they are not primarily developed as complete cross platform applications for ordinary users. This shows that there is still a need for systems that combine AI prediction with broader software features and user centered design.

AgroAI addresses this gap by being designed as a cross platform system that supports both web and mobile usage. This gives it broader accessibility compared with solutions that are available only through one medium. Users can interact with the system through a mobile application when they need convenient access from the field, and they can also use the web application for a larger interface and easier record viewing. This dual platform approach makes AgroAI more practical because it supports different user preferences and usage environments. It also reflects the current direction of modern software systems, where users expect services to be available across multiple devices.

A key differentiator of AgroAI is the integration of modern full stack technologies with an AI inference layer. The implementation of AgroAI uses Flutter for the mobile application, MERN stack with TypeScript for the web application, MongoDB for data storage, and a trained .h5 plant disease detection model deployed on Hugging Face Spaces for live inference. This provides a cleaner and more practical architecture for a final year project because it separates the major responsibilities of the system. The frontend handles user interaction, the backend manages request processing and business logic, the database stores user and diagnosis records, and the AI inference layer performs disease classification. This separation improves maintainability, modularity, and future extension.

In comparison with many existing systems, AgroAI is also designed with user accessibility and record management in mind. The project does not only provide a one time prediction result. It also supports diagnosis history, disease trends, scan calendar, saved farm boundary visualization, and profile management. These features increase the usefulness of the system because plant disease diagnosis is often an ongoing activity. Users may need to review previous results, observe repeated disease patterns, or track scan dates over time. By including these supporting features, AgroAI becomes more than a basic image prediction tool and moves closer to a practical crop health monitoring platform.

The use of a live inference environment is another important strength of AgroAI. In many research projects, trained models remain offline and are tested only in controlled environments. Although such models may provide good results during experimentation, they do not always represent a complete user facing system. AgroAI uses Hugging Face Spaces to deploy the model so that the system can perform real time prediction as part of the application workflow. This means the user can upload an image, the backend can forward it to the AI model, and the

prediction result can be returned and displayed through the application. This makes the AI component operational rather than only theoretical.

The analysis also shows that usability is a major factor in the success of agricultural technology. Even if a system has a strong model, users may not benefit from it if the interface is difficult or confusing. AgroAI therefore focuses on providing a simple and understandable user experience. Features such as registration, login, image upload, diagnosis result display, history, trends, calendar, and map are organized to make the system easier to use. This focus on usability is important because the target users may include people who are not highly technical. A clear interface helps users complete the disease diagnosis process without unnecessary difficulty.

Overall, the analysis shows that AgroAI is aligned with the broader direction of current plant disease detection systems, but it is shaped more specifically around accessibility, integration, and practical software delivery. The project does not simply repeat existing ideas; instead, it combines them into a structured cross platform solution supported by modern technologies and AI based prediction. By connecting web and mobile interfaces, backend services, MongoDB storage, and live AI inference, AgroAI provides a complete and usable system for plant disease diagnosis.

1.6 Methodology and Software Lifecycle

AgroAI development is based on Agile approach along with iterative software process model. This method is appropriate for the project since the system comprises of several interconnected elements, such as the web platform, mobile application, backend services, the database, and the artificial intelligence module. These parts can only be designed gradually and implemented, tested and improved, thus a flexible and incremental development process proves to be a better solution rather than a linear one.

With Agile approach, the project can be built into smaller blocks, and one can work on the project block by block. This ensures that one can easily introduce the system gradually, assess the result of each step and correct where needed. The iterative lifecycle is especially relevant in a project such as AgroAI since it facilitates the gradual incorporation of different technologies and the development team is also able to improve the system as time progresses.

The system can be enhanced by successive development and testing instead of trying to put everything in it at once.

Such methodological approach is useful too as software projects may change in implementation. The actual working system is the final system in the case of AgroAI, which mirrors the applied choices in real development. In practice, this meant that we did not wait until all components were built before testing how they worked together. Each sprint ended with a working, testable version of some part of the system whether that was the authentication flow, the image upload process, or the connection between the backend and the AI inference service. This approach helped us catch integration issues early, rather than discovering them at the end when the entire system was already built and changes would have been far more costly.

1.6.1 Rationale behind Selected Methodology

The selection of Agile methodology and an iterative software lifecycle is justified by the nature and structure of the AgroAI project. AgroAI is not a simple single module application; it combines several technical layers such as frontend development, backend services, database management, user authentication, image upload handling, and AI based plant disease prediction. These components are connected with each other, but each component also has its own separate development requirements. Due to this reason, it is more practical to develop the system in stages rather than following a single fixed development sequence. Agile methodology supports this kind of gradual development because it allows each module to be planned, implemented, tested, and improved before being fully integrated into the complete system.

One of the main reasons for selecting Agile methodology is the modular nature of AgroAI. The system includes a mobile application developed using Flutter, a web application developed using the MERN stack with TypeScript, a backend developed with Node.js and Express.js, MongoDB database integration, and an AI model deployed on Hugging Face Spaces. Each of these parts requires separate attention during development. For example, the frontend must be designed to provide a simple user interface, the backend must handle requests and responses correctly, the database must store diagnosis records properly, and the AI model must return prediction results through live inference. Agile methodology makes it easier to work on these parts step by step and improve them as required.

The flexibility Agile provides during development also justifies its selection for this project. In many software projects, all requirements are not completely clear at the beginning. Some features, design decisions, or technical details become clearer after partial implementation and testing. The same situation applies to AgroAI, where features such as image upload, diagnosis history, disease trends, calendar records, and farm map visualization require continuous refinement. An iterative lifecycle allows these improvements to be made without disturbing the entire project structure. If a certain feature needs modification, the development team can revise that module in the next iteration instead of redesigning the whole system.

Agile methodology is also suitable for AgroAI because the project depends on integration between different technologies. The frontend, backend, database, and AI inference service must communicate smoothly for the system to work properly. When a user uploads a plant leaf image, the frontend sends the image to the backend, the backend validates and forwards it to the deployed AI model, the model returns the prediction, the backend stores the result in MongoDB, and the frontend displays the output to the user. This complete process cannot be tested properly only at the end of development. It needs repeated testing at different stages. Agile supports this by allowing continuous testing and integration during each development cycle.

Continuous feedback is another important reason for using Agile methodology. In AgroAI, user experience is an important concern because the system must be easy to use for people who may not have advanced technical knowledge. Features such as registration, login, image upload, result display, history, trends, calendar, and profile management must be clear and understandable. Through iterative development, feedback can be collected after implementing each feature, and necessary changes can be applied in the next cycle. This helps improve usability and makes the final system closer to user expectations.

The iterative lifecycle also reduces development risk. If the complete system is developed in one long sequence and tested only at the end, major issues may appear late and become difficult to fix. In AgroAI, this could include problems such as incorrect API communication, database storage errors, image upload failure, or delay in receiving prediction results from the AI model. By using Agile, such issues can be identified earlier because each module is tested after development. Early detection of problems helps improve system stability and prevents small issues from becoming major failures.

Agile methodology also supports better project management. Since AgroAI includes multiple features and technologies, the development process needs proper planning and organization. Agile allows the project to be divided into smaller tasks or iterations. For example, one iteration may focus on user authentication, another on image upload, another on AI model integration, and another on diagnosis history or dashboard features. This makes the project easier to manage because each part has a clear development goal. It also helps in tracking progress and ensuring that important modules are completed in an organized manner.

The selected methodology also supports future improvement. AgroAI is designed as a practical AI powered crop disease diagnosis system, and it can be extended in the future with additional features such as more crop disease classes, improved model accuracy, farmer advisory support, multilingual interface, notification system, or advanced analytics. Since Agile encourages continuous improvement and modular development, the system can be expanded without changing its complete structure. This makes the methodology suitable not only for current development but also for future enhancement of the project.

Overall, Agile methodology and the iterative software lifecycle are appropriate for AgroAI because they support flexibility, modular progress, continuous testing, user feedback, and practical system integration. The selected methodology helps the project move from concept to implementation in a controlled and organized manner. It allows each technical layer to be developed and refined separately while ensuring that all components work together as one complete cross platform AI powered plant disease diagnosis system.

2 Problem Definition

2.1 Problem Statement

Agriculture is highly dependent on plant health, and plant diseases are among the major factors that reduce crop quality and productivity. When diseases are not identified at an early stage, they can spread rapidly and damage a large portion of the crop, leading to financial loss and reduced agricultural output. In many cases, disease identification is still performed through manual observation or by consulting agricultural experts. Although these methods may be useful, they are often slow, inconsistent, and not easily accessible to every user. This creates

difficulty in obtaining timely and accurate diagnosis, especially when fast action is required to prevent further damage.

Traditional disease diagnosis methods also present an important issue, as they often depend on visible symptoms and personal experience. Plant diseases often show similar visual patterns, which can increase the chance of misdiagnosis. A user who does not have technical or agricultural expertise may find it difficult to distinguish between healthy and infected plants or between different types of diseases. As a result, decisions taken on the basis of uncertain diagnosis may be ineffective and may further affect plant health and crop yield.

The problem becomes more significant when considering the lack of accessible digital tools that combine plant disease detection with modern software delivery. Existing solutions are often limited in accessibility, restricted to a single platform, or not designed as complete user oriented systems. Therefore, there is a need for a cross platform software solution that can help users identify plant diseases through a simpler, faster, and more structured process. AgroAI is developed to address this problem by providing an AI powered plant disease diagnosis system that allows users to upload plant leaf images and receive disease related predictions through web and mobile applications.

2.2 Deliverable and Development Requirements

The ultimate product of the AgroAI is an entirely cross platform plant disease diagnosis system that will serve web and mobile users. This system will aim at offering a combined software platform whereby users can communicate with the plant disease detection in a practical and accessible manner. The deliverables of the final project are thus the mobile application, web platform, backend services, database integration, and the artificial intelligence inference support.

The Flutter framework is used to develop the mobile application, which offers a special mobile interface to users wishing to access the system via handheld gadgets. The web platform is built upon the MERN stack with TypeScript, with React with TypeScript on the front end, Node.js with Express.js as the backend services, and MongoDB as data storage and management in the system. Moreover, the project contains an artificial intelligence component, which consists of a trained model, which is stored in the form of h5 and deployed on Hugging Face Spaces to

facilitate live prediction and inference. The combination of these parts is the primary technical project deliverables.

In terms of function, the project should provide a system in which users can upload or take pictures of plant leaf, submit them to be analyzed, and get the prediction of plant diseases made by the AI model. It is also anticipated that the system will keep user related information and diagnosis related information in an orderly manner. Because the project is created as a software solution, and not a theoretical study, the following requirements are needed to develop it: the proper integration of the frontend, the backend, database, and AI inference layer. It also demands that the system should be accessible, maintainable and usable by the actual users.

AgroAI also involves a more organized software process in order to make sure that the end product is finished and cohesive. This involves requirement analysis, design planning, implementation, testing and refining of the project modules. As the project will involve several parts, efficient communication between the web and mobile interfaces, server side processing, data storage, and AI prediction service are crucial to implementing a successful project. Thus, the project deliverables do not only include the source code, but also the entire working system itself and its documentation.

2.3 Current System

Currently, identification of plant diseases in most instances still relies on traditional and manual procedures. Farmers and growers tend to examine the leaves of plants with their bare eyes and attempt to decide whether a plant is healthy or diseased based on visible symptoms alone. In more severe or uncertain cases, they may consult an agricultural professional, visit a local extension office, or search general internet sources to compare observed symptoms with known disease patterns. Such methods can be effective to some degree, but they are not consistently reliable. The visible symptoms of many plant diseases can appear similar to one another, making accurate identification through unaided observation difficult even for experienced growers. Furthermore, the availability of agricultural experts is not guaranteed, particularly for smallholder farmers in rural or remote areas who may not have convenient access to professional guidance.

The existing approach also suffers from the absence of a straightforward and unified digital procedure for diagnosing plant diseases. Without a dedicated software platform, users are left

to rely on scattered sources of information, subjective visual evaluation, or delayed expert responses. This makes the overall process less efficient and reduces the likelihood of timely intervention, which is critical because plant diseases can spread rapidly across crops if not addressed promptly. Late or inaccurate diagnosis can result in reduced yield quality, significant financial loss, and in severe cases, complete crop failure.

Beyond the diagnostic limitations, conventional methods also fail to provide any structured mechanism for recording, storing, or reviewing diagnosis related information. A farmer who identifies a disease manually has no systematic way to log that observation, track its recurrence, or monitor disease trends over time. Each diagnosis remains an isolated event with no connection to previous occurrences or broader patterns of disease activity. This lack of organized record keeping makes it difficult for farmers to make informed decisions about crop management, treatment history, or seasonal disease behavior.

Additionally, the existing approach offers no cross platform accessibility. Farmers and agricultural users increasingly interact with digital tools through both mobile devices and web browsers, yet traditional methods of disease identification do not translate into any consistent digital workflow. There is no unified environment where a user can capture or upload a plant image, receive a structured prediction, review past diagnoses, and monitor disease activity from the same platform across different devices.

The existing state of affairs therefore highlights a clear need for a more modern and technology driven solution. An AI based cross platform system such as AgroAI directly addresses these shortcomings by embedding the plant disease diagnosis process within a structured software environment. Rather than depending on unaided observation or uncertain external advice, users can upload a plant leaf image, receive an AI generated disease prediction, and interact with a complete set of supporting features including diagnosis history, calendar based scan records, farm boundary mapping, disease trend monitoring, and profile management. By consolidating these capabilities into a single accessible platform available on both web and mobile, AgroAI offers a significantly more reliable, efficient, and organized alternative to the manual and fragmented methods that currently dominate plant disease identification in agricultural practice.

3 Requirement Analysis

3.1 Use Case Diagram

Requirement analysis is one of the most important stages in software development because it defines what the system is expected to do and how users will interact with it. In AgroAI, the requirement analysis is centered on the practical activities that users perform while using the system, such as creating an account, uploading plant images, receiving plant disease predictions, and viewing previous diagnosis records. Since the system is designed as a cross platform plant disease diagnosis solution, the requirements must reflect both functional interaction and the overall user flow across the web and mobile environments. The project template expects this chapter to include use case diagrams, detailed use cases, functional requirements, and non functional requirements.

A use case diagram provides a high level representation of the system by showing the main actors and the actions they can perform. For AgroAI, the use case model helps in identifying the boundaries of the system and clarifies how different users interact with the plant disease diagnosis platform. Based on the actual project direction and implemented scope, the system mainly involves two actors: the **User** and the **Administrator**. The User is the primary actor who interacts with the application for plant disease diagnosis and record viewing, while the Administrator manages system level oversight functions where applicable.

The **User** actor performs the main functions of the system. These include **registration, login, leaf image upload, viewing diagnosis results, viewing diagnosis history, viewing disease trends, viewing the farm on the map, viewing scan dates in the calendar, managing the user profile, and logout**. The **Administrator** actor is associated with system level functions such as **viewing system users, viewing diagnosis records, and monitoring overall system activity**. The use case structure therefore reflects the practical flow of the application and highlights the major services that **AgroAI** provides to its users, as shown in **Figure 3.1**.

For the purpose of the report, the use case diagram of **AgroAI** includes the following main use cases:

Under the User actor:

- Register Account
- Login to System

- Upload Leaf Image
- View Diagnosis Result
- View Diagnosis History
- View Disease Trends
- View Farm on Map
- View Scan Dates in Calendar
- Manage User Profile
- Logout from System

Under the Administrator actor:

- View System Users
- View Diagnosis Records
- Monitor System Activity

The relationship between these use cases can also be explained logically. For example, **View Diagnosis Result** depends on **Upload Leaf Image**, because the system cannot generate a diagnosis unless an image is first submitted. Similarly, **View Diagnosis History**, **View Disease Trends**, **View Farm on Map**, **View Scan Dates in Calendar**, and **Manage User Profile** require the user to be logged into the system. Thus, the use case diagram shown in **Figure 3.1** provides a clear understanding of the system's main interactions and serves as a basis for the detailed use cases in the following sections.

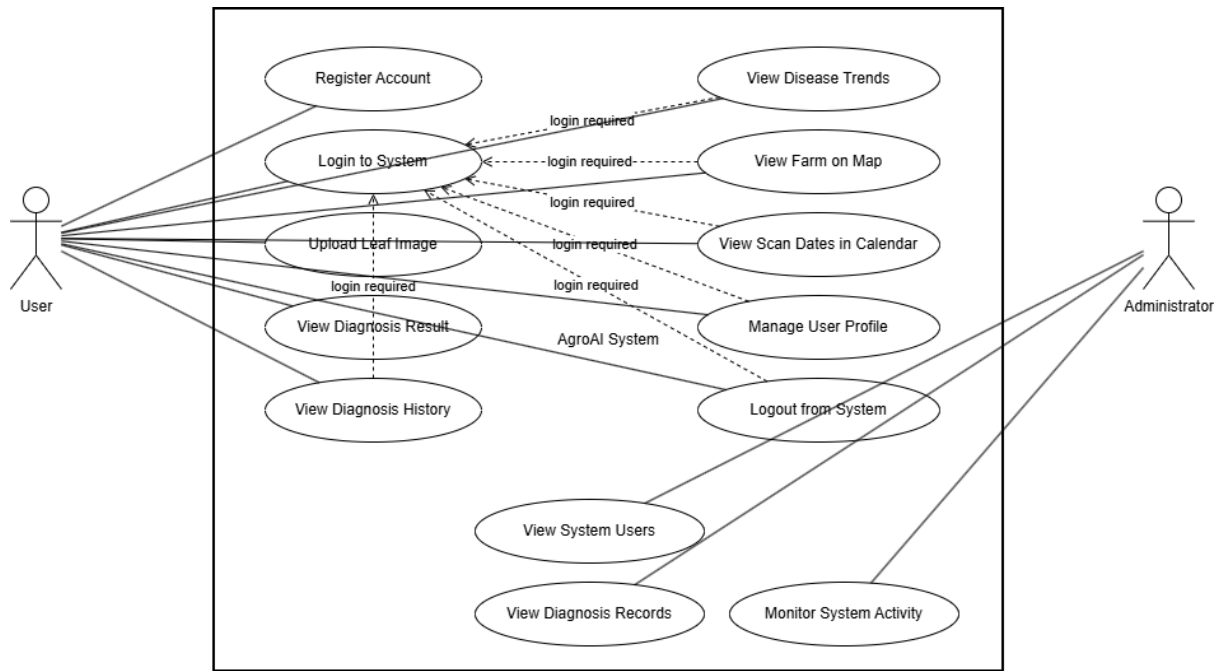


Figure 3-1 Use Case Diagram of AgroAI

3.2 Detailed Use Cases

Detailed use cases explain the step by step interaction between the actor and the system. They describe how each major system function begins, what conditions must be satisfied before it starts, what actions occur during execution, and what the expected outcome is. This level of detail is useful because it converts broad system functionality into clearly understandable behavior. For AgroAI, the most important detailed use cases are those directly linked to the actual implemented system: user registration, user login, leaf image upload for disease detection, viewing disease diagnosis results, viewing history, and profile management.

3.2.1 Use Case: Register Account

This use case allows a new user to create an account in the AgroAI system by providing the required registration information. It is the starting point for authenticated access and enables the user to interact with protected system features such as diagnosis history, disease trends, farm records, calendar, and profile management. The registration process must be simple, validated, and secure so that only correct and unique account information is stored in the system.

Table 3.2.1 Use Case Register Account

Element	Description
Use Case ID	UC 1
Use Case Name	Register Account
Primary Actor	User
Description	Allows a new user to create an account in the system
Trigger	User selects the sign up option
Preconditions	User is not already logged in
Postconditions	New user account is created and stored
Normal Flow	Open registration page → Enter details → Validate data → Create account → Show success message
Alternative Flow	Duplicate email, invalid input, incomplete fields
Business Rules	Email must be unique, and required fields must be filled

3.2.2 Use Case: Login to System

This use case allows a registered user to access the system using valid login credentials. It is required for protected features of the application, including diagnosis history, disease trends, farm map viewing, calendar access, and profile management. A successful login confirms the identity of the user and allows the system to provide personalized access to records and services.

Table 3.2.2 Use Case Login to System

Element	Description
Use Case ID	UC 2
Use Case Name	Login to System

Primary Actor	User
Description	Allows a registered user to log in with valid credentials
Trigger	User submits login form
Preconditions	User account already exists
Postconditions	User is authenticated and redirected to dashboard
Normal Flow	Open login page → Enter email/password → Validate → Authenticate → Redirect
Alternative Flow	Invalid credentials, incomplete fields
Business Rules	Only registered users can log in

3.2.3 Use Case: Upload Leaf Image

This use case represents the core functionality of AgroAI. It allows the user to upload or capture a plant leaf image so that the system can analyze it for disease prediction. The quality of this interaction is important because the diagnosis process begins from image submission. The system must support valid image upload, proper file handling, and smooth transfer of the image to the backend and AI inference environment.

Table 3.2.3 Use Case Upload Leaf Image

Element	Description
Use Case ID	UC 3
Use Case Name	Upload Leaf Image
Primary Actor	User
Description	Allows user to upload or capture plant leaf image for analysis
Trigger	User selects disease detection feature
Preconditions	User is logged in

Postconditions	Image is sent for AI based analysis
Normal Flow	Open upload screen → Select/capture image → Validate file → Send to backend → Forward for inference
Alternative Flow	Invalid image, upload failure, inference unavailable
Business Rules	Only supported image types are accepted

3.2.4 Use Case: View Diagnosis Result

This use case allows the user to view the prediction returned by the AI model after the uploaded image has been processed. The system should display the disease label and related output in a clear and understandable format so that the result becomes meaningful to the user. The diagnosis result is one of the most important outputs of the entire system.

Table 3.2.4 Use Case View Diagnosis Result

Element	Description
Use Case ID	UC 4
Use Case Name	View Diagnosis Result
Primary Actor	User
Description	Allows user to view AI based disease prediction result
Trigger	AI analysis completes
Preconditions	Valid image uploaded and processed
Postconditions	Diagnosis result displayed and may be saved in history
Normal Flow	AI returns prediction → Backend receives result → Save record → Display result
Alternative Flow	Low confidence, uncertain result, system communication failure

Business Rules	Result must be shown clearly and linked to the uploaded image
----------------	---

3.2.5 Use Case: View Diagnosis History

This use case allows the authenticated user to access previous diagnosis records stored in the system. It increases the practical value of AgroAI by allowing users to revisit earlier results instead of treating the system as a one time prediction tool only. The system must retrieve only the records associated with the logged in user.

Table 3.2.5 Use Case View Diagnosis History

Element	Description
Use Case ID	UC 5
Use Case Name	View Diagnosis History
Primary Actor	User
Description	Allows user to view previous diagnosis records
Trigger	User opens history section
Preconditions	User is logged in and has stored records
Postconditions	Diagnosis history is displayed
Normal Flow	Open history page → Fetch user records → Display structured list
Alternative Flow	No records found, data loading failure
Business Rules	User can only access own history

3.2.6 Use Case: View Disease Trends

This use case allows the authenticated user to view disease trend information based on stored scan records. It helps the user monitor the frequency and pattern of plant diseases over time. The system should present this information in a clear visual format so that it supports better understanding and decision making.

Table 3.2.6 Use Case View Disease Trends

Element	Description
Use Case ID	UC 6
Use Case Name	View Disease Trends
Primary Actor	User
Description	Allows user to view disease trend patterns from stored records
Trigger	User opens disease trends section
Preconditions	User is logged in and trend related data is available
Postconditions	Disease trend data is displayed
Normal Flow	Open trends page → Fetch user scan data → Process records → Display trend charts or summaries
Alternative Flow	No trend data available, data loading failure
Business Rules	Trend data must be based only on the authenticated user's records

3.2.7 Use Case: View Farm on Map

This use case allows the authenticated user to view farm boundaries and related farm information on a map interface. It supports spatial monitoring by helping the user visually identify the farm area connected to disease scans and crop records. The system must display only the farm data associated with the logged in user.

Table 3.2.7 Use Case View Farm on Map

Element	Description
Use Case ID	UC 7
Use Case Name	View Farm on Map
Primary Actor	User

Description	Allows user to view farm boundaries and location information on a map
Trigger	User opens the farm map section
Preconditions	User is logged in and farm data is available
Postconditions	Farm information is displayed on the map
Normal Flow	Open map section → Fetch farm data → Load map → Display boundaries and related information
Alternative Flow	No farm data found, map loading failure
Business Rules	User can only view farm data associated with their own account

3.2.8 Use Case: View Scan Dates in Calendar

This use case allows the authenticated user to view scan related dates in a calendar interface. It helps the user track when plant scans were performed and supports organized monitoring of crop health activity over time. The system should present scan records according to their saved dates.

Table 3.2.8 Use Case View Scan Dates in Calendar

Element	Description
Use Case ID	UC 8
Use Case Name	View Scan Dates in Calendar
Primary Actor	User
Description	Allows user to view scan dates in calendar form
Trigger	User opens the calendar section
Preconditions	User is logged in and scan records exist
Postconditions	Scan dates are displayed in the calendar

Normal Flow	Open calendar page → Fetch scan records → Match records with dates → Display in calendar view
Alternative Flow	No scan records found, calendar loading failure
Business Rules	Calendar data must show only the authenticated user's scan records

3.2.9 Use Case: Manage User Profile

This use case allows the authenticated user to view and update personal account information. It supports personalization and account maintenance by enabling the user to modify selected profile fields when required. The system must validate the updated information before saving it.

Table 3.2.9 Use Case Manage User Profile

Element	Description
Use Case ID	UC 9
Use Case Name	Manage User Profile
Primary Actor	User
Description	Allows user to view and update profile information
Trigger	User opens profile section
Preconditions	User is logged in
Postconditions	Profile updates are stored successfully
Normal Flow	Open profile page → View details → Edit fields → Validate → Save
Alternative Flow	Invalid data, save failure
Business Rules	Changes must apply only to the authenticated user

3.2.10 Use Case: Logout from System

This use case allows the authenticated user to securely exit the AgroAI system. It ends the current session and prevents unauthorized access to protected features after the user leaves the application.

Table 3.2.10 Use Case Logout from System

Element	Description
Use Case ID	UC 10
Use Case Name	Logout from System
Primary Actor	User
Description	Allows user to securely log out from the system
Trigger	User selects the logout option
Preconditions	User is logged in
Postconditions	User session is terminated and access to protected features is removed
Normal Flow	Click logout → End session → Redirect to public page or login screen
Alternative Flow	Session timeout, logout failure
Business Rules	Logout must clear the authenticated session securely

3.3 Functional Requirements

Functional requirements describe the specific services and behaviors that the system must provide. These requirements define what AgroAI is expected to do from the perspective of system functionality. Since AgroAI is an AI powered plant disease diagnosis platform, its functional requirements must support user management, disease prediction, record maintenance, and interaction between the frontend, backend, database, and AI inference service.

3.3.1 Description of Functional Requirements

The functional requirements of AgroAI cover all major operations that users perform during interaction with the system. These include creating an account, logging in, uploading leaf images, receiving disease predictions, viewing stored history, and updating profile information. In addition, administrative monitoring may also be included where applicable. These requirements ensure that the system fulfills its intended purpose as a complete plant disease diagnosis platform rather than a single isolated prediction service.

To present these requirements in a formal and organized manner, they should be summarized in tabular form. This makes them easier to read, reference, and validate during the later stages of design, implementation, and testing.

Table 3.3.1 Functional Requirements of AgroAI

Req ID	Functional Requirement	Priority
FR 1	The system shall allow a new user to register an account	High
FR 2	The system shall validate registration information before account creation	High
FR 3	The system shall allow registered users to log in using valid credentials	High
FR 4	The system shall restrict protected features to authenticated users only	High
FR 5	The system shall allow users to upload or capture a plant leaf image	High
FR 6	The system shall validate uploaded images before processing	High
FR 7	The system shall send uploaded images to the backend for AI based prediction	High
FR 8	The system shall display the disease diagnosis result to the user	High
FR 9	The system shall store diagnosis results in the database	High
FR 10	The system shall maintain diagnosis history for each user	High
FR 11	The system shall allow users to view their own diagnosis history	High
FR 12	The system shall allow users to view and update profile information	Medium

FR 13	The system shall allow authenticated users to log out securely	Medium
FR 14	The system may support administrative monitoring of users and records	Medium

3.4 Non Functional Requirements

Non functional requirements describe the quality characteristics of the system. Unlike functional requirements, which define what the system does, non functional requirements define how well the system performs its functions. In AgroAI, these requirements are especially important because the system must remain usable, responsive, secure, reliable, and maintainable across both web and mobile platforms.

3.4.1 Description of Non Functional Requirements

The non functional requirements of AgroAI focus on the overall quality and dependability of the platform. Since the system is designed to be used through both web and mobile interfaces, qualities such as usability, portability, and performance are highly important. Similarly, because the platform handles user accounts and personal diagnosis history, security and reliability are also essential. A maintainable and scalable system is important for future development and improvement.

Table 3.4.1 Non Functional Requirements of AgroAI

Req ID	Non Functional Requirement	Description
NFR 1	Usability	The system should provide a clear, simple, and user friendly interface
NFR 2	Performance	The system should respond to user requests within a reasonable time
NFR 3	Reliability	The system should perform consistently under normal operating conditions
NFR 4	Security	Protected data and features should only be accessible to authorized users

NFR 5	Maintainability	The system should support easy updates, debugging, and future improvements
NFR 6	Portability	The system should function across both web and mobile environments
NFR 7	Scalability	The system should support future growth in users, records, and features
NFR 8	Availability	The system should remain accessible whenever connected services are operational

4 Design and Architecture

4.1 System Architecture

The design and architecture of AgroAI define the overall structure of the system and explain how its major components interact to perform plant disease diagnosis in a reliable and organized way. Since AgroAI is developed as a cross platform platform, its architecture supports smooth communication between the user facing applications, backend services, database, and artificial intelligence inference layer. A proper architectural design is important because it ensures that each system component performs a clear and specific role while remaining connected to the others in a consistent manner. In software projects such as AgroAI, architecture is not only concerned with technology selection; it also defines how the complete platform is organized to achieve usability, maintainability, and efficient flow of information.

AgroAI follows a modular client server architecture. In this architectural style, the user does not interact directly with the data layer or with the artificial intelligence model. Instead, all communication passes through a central backend service that acts as the coordinating layer of the platform. This arrangement suits AgroAI because the project contains multiple functional concerns, including account management, image upload, disease prediction, history maintenance, and profile related access. If all these responsibilities are handled in a single monolithic structure without separation, the system becomes harder to manage, update, and secure. A modular architecture solves this problem by dividing the system into components with well defined responsibilities.

The architecture of AgroAI can be understood through four main layers: the presentation layer, the application layer, the data layer, and the AI inference layer. Each of these layers is essential to the complete operation of the platform. The presentation layer provides the user interface. The application layer controls request processing and business logic. The data layer stores persistent information. The AI inference layer performs the disease classification task. The practical benefit of this separation became clear when we updated the disease detection model mid development. Because the model sits entirely within the inference layer and communicates with the backend only through a defined API call, we were able to swap in a retrained version without modifying a single line of frontend or backend code. This kind of clean boundary between layers is easy to describe in a design document, but it only feels real once you have needed to make a change across a live system.

The presentation layer of AgroAI consists of two main user facing applications: the mobile application and the web application. The mobile side is developed in Flutter, while the web side is implemented using the MERN stack with TypeScript, with React and TypeScript used for building the web interface. The purpose of this layer is to provide a simple, clear, and accessible way for users to interact with the system. Through this layer, users create accounts, log in, upload plant leaf images, view diagnosis results, access previous records, and update profile details. The design of the presentation layer is important because AgroAI is intended to be used by users who may not necessarily have strong technical knowledge. Therefore, the interfaces remain understandable, visually structured, and easy to navigate. In architectural terms, this layer is responsible only for collecting input and displaying output; it does not perform core disease analysis itself. Instead, it depends on backend services for actual processing.

The application layer is implemented using Node.js with Express.js. This layer acts as the main controller of the system. It receives requests from both the web and mobile applications, validates them, performs authentication checks, communicates with the database, and forwards disease detection requests to the AI inference service. In practical terms, when a user uploads a plant image, that image first reaches the backend. The backend verifies the request, prepares the required input, and sends it to the AI model. When the model returns a disease prediction and confidence score, the backend processes that response, saves the relevant record in the database, and returns structured output to the frontend. In this way, the backend acts as the bridge between user interaction and intelligent prediction. Because the application layer is

responsible for business logic, routing, and service coordination, it is central to the operation of the entire platform.

The data layer is implemented using MongoDB. This layer stores all persistent information necessary for the platform to function properly. Such information includes user account data, diagnosis related records, image references, and history entries. The importance of the data layer lies in the fact that AgroAI is not just a one time disease prediction tool. It is a complete platform in which users return to the system, access their previous diagnosis results, and manage their profiles. For this reason, the data layer ensures that records are stored in a way that is structured, retrievable, and securely associated with the correct user. Since AgroAI uses account based interaction, every diagnosis record links to the user who generates it. This makes the database essential for authentication support, history maintenance, and long term record management.

The AI inference layer is the intelligent part of the architecture. It is responsible for analyzing plant leaf images and returning disease classification results. In AgroAI, the disease detection model is stored in .h5 format and deployed on Hugging Face Spaces for live inference. This layer receives image input from the backend, processes the image according to the trained deep learning model, and returns a prediction response containing the disease class and related confidence value. The AI inference layer is intentionally separated from the rest of the platform because this makes the system more modular. It allows the machine learning model to remain a specialized component with one clear responsibility: disease classification. This separation also improves maintainability because changes to the model, such as retraining or replacement, can be managed without directly affecting frontend design or general backend structure.

From an architectural perspective, the communication among these layers follows a structured path. A request originates in the presentation layer when the user performs an action. That request moves to the application layer, where logic and validation are applied. When data storage or retrieval is needed, the application layer interacts with the data layer. When plant disease prediction is required, the application layer communicates with the AI inference layer. Once a response is received, the application layer sends structured output back to the presentation layer for display to the user. This controlled communication path ensures that each component handles only the tasks for which it is designed, thereby improving system organization and reducing unnecessary complexity.

The layered structure of AgroAI was not chosen arbitrarily it directly addresses practical concerns that came up during development. Separating the AI inference layer from the backend, for example, meant that when we needed to retrain the model or adjust its deployment on Hugging Face Spaces, we could do so without touching the backend logic or the frontend code. Similarly, serving both the web and mobile applications from a single backend meant we only had to maintain one set of API endpoints rather than building and debugging two separate server side implementations. Maintainability, portability, and scalability are not just theoretical qualities of this architecture they translated into real development decisions that saved time and reduced the risk of introducing bugs. For a final year project, this architectural design is suitable because it demonstrates a structured and realistic approach to full stack software development combined with intelligent services.

Thus, the overall architecture of AgroAI can be summarized as follows:

- Presentation Layer: Flutter mobile application and MERN based web application with TypeScript
- Application Layer: Node.js and Express.js backend services
- Data Layer: MongoDB database for persistent storage
- AI Inference Layer: .h5 plant disease classification model deployed on Hugging Face Spaces

This layered structure gives AgroAI a stable foundation for cross platform use, secure record management, and AI powered plant disease prediction.

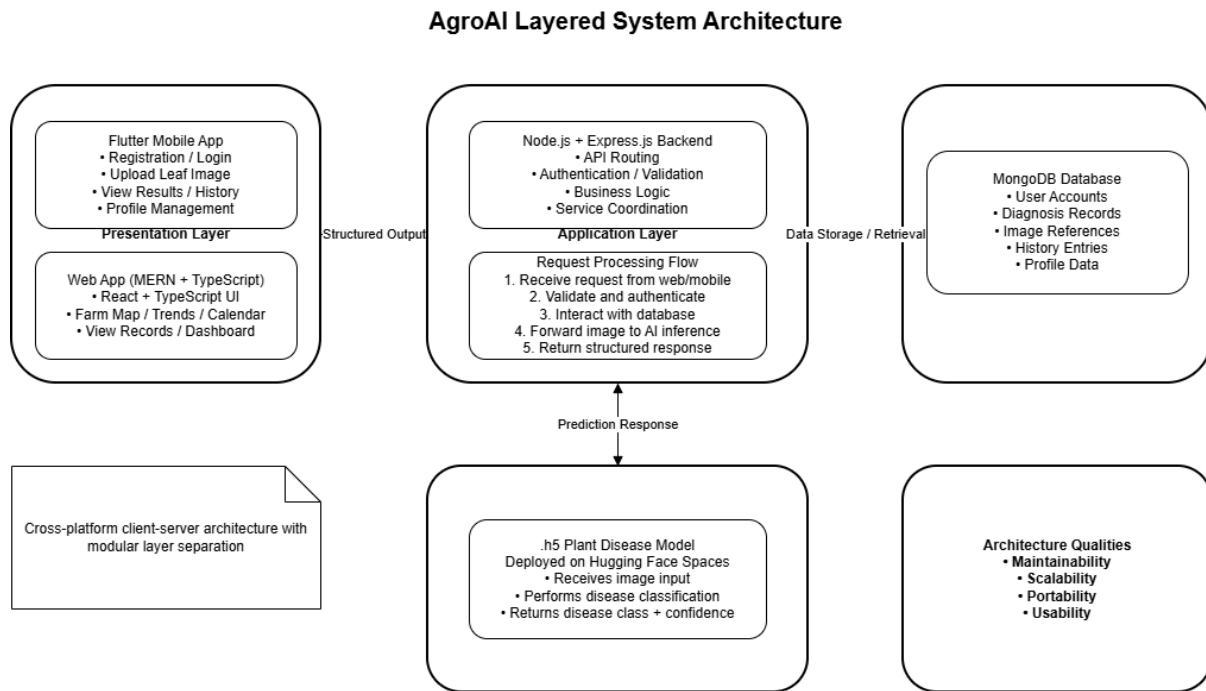


Figure 4-1 System Architecture of AgroAI

4.2 Data Representation

Data representation explains how system information is structured, stored, and related internally. In AgroAI, proper data representation is essential because the platform depends on several forms of information that remain connected in a meaningful way. These include user information, uploaded images, diagnosis outputs, and history records. Without a clear data design, the system cannot authenticate users properly, connect uploaded images with diagnosis results, or maintain personalized history for future viewing. Therefore, data representation is one of the core aspects of the design and architecture of AgroAI.

The data design of AgroAI is centered around a small number of important entities. These are the **User**, **Image**, **Diagnosis**, **AI Model**, **History**, and supporting authentication related structures. Each entity supports a specific functional requirement of the system. The User entity supports account based access, the Image entity represents uploaded plant leaves, the Diagnosis entity stores prediction results, the History entity supports retrieval of earlier records, and the AI Model entity represents the system component responsible for image analysis. When these entities are properly linked, they form the internal informational structure that makes the platform functional and organized.

The **User** entity is the most fundamental data entity because nearly all personalized features of the system depend on it. A user account stores identifying and authentication related information such as name, email, password related values, role, and profile details. Through this entity, AgroAI determines who is currently interacting with the system and which stored records belong to that person. The User entity is also important because it provides the reference point for diagnosis history, image ownership, and access control. In a platform like AgroAI, the relationship between the user and stored records remains clear and secure, which makes the User entity central to the overall data model.

The **Image** entity stores information related to the plant leaf image uploaded by the user. Since plant disease prediction begins with image submission, this entity acts as the primary input representation of the disease detection workflow. Important fields in this entity include image identifier, user reference, file path or URL, upload time, and status. The Image entity ensures that there is a stored link between the visual input submitted by the user and the diagnosis output later produced by the AI model. This is important because the system does not treat the diagnosis as an isolated result; instead, the result remains connected to the exact image that generates it. Such linkage improves traceability and helps support later review of records.

The **Diagnosis** entity represents the result of the AI based disease analysis. It stores information such as the predicted disease label, confidence or accuracy score, creation time, and references to both the image and the user. This entity is particularly important because it is the direct output of the core function of AgroAI. When the AI model classifies a plant leaf image, the resulting disease information must be saved in a form that can be displayed immediately and later retrieved again. The Diagnosis entity therefore serves both short term and long term purposes. In the short term, it supports result display after prediction. In the long term, it supports history maintenance and record review. Because of this, the Diagnosis entity sits at the center of the link between uploaded images, user accounts, and intelligent model output.

The **AI Model** entity is conceptual rather than purely record based, but it remains part of the data representation discussion because it transforms data from one state into another. It receives the uploaded image data as input, processes it through the trained deep learning model, and produces the diagnosis result as output. Although the AI model itself is not stored in the database like ordinary user or diagnosis records, it is part of the information structure of the system because it determines how image data becomes plant disease knowledge. In the design

representation, it can therefore be shown as a key system component connected to image input and diagnosis output.

The **History** entity maintains the connection between users and their previous diagnosis results. While history may be represented directly through diagnosis records linked to a user, it is still useful conceptually to treat history as its own design element because it emphasizes the platform’s ability to preserve and retrieve earlier interactions. This entity supports one of the practical benefits of AgroAI: users return to the system and revisit previous diagnoses instead of treating the platform as a temporary prediction tool only. In a plant disease diagnosis platform, this is useful because users want to compare symptoms over time, review earlier outcomes, or maintain a record of plant health changes.

The relationship among these entities can be described in a clear way. One **User** may upload many **Images**. One uploaded **Image** leads to one or more **Diagnosis** records depending on how the result is represented. One **User** may therefore also have many **Diagnosis** records. The **History** view is essentially derived from this relationship, because it shows the collection of diagnosis results associated with a specific user over time. The **AI Model** takes image input and generates diagnosis output, connecting the input side of the system with the result side of the system. This design makes AgroAI a user centered data system in which all important diagnostic information remains connected to the initiating account.

For formal academic presentation, this section includes a **data representation diagram** and a **data dictionary table**. The diagram visually shows the main entities and their relationships.

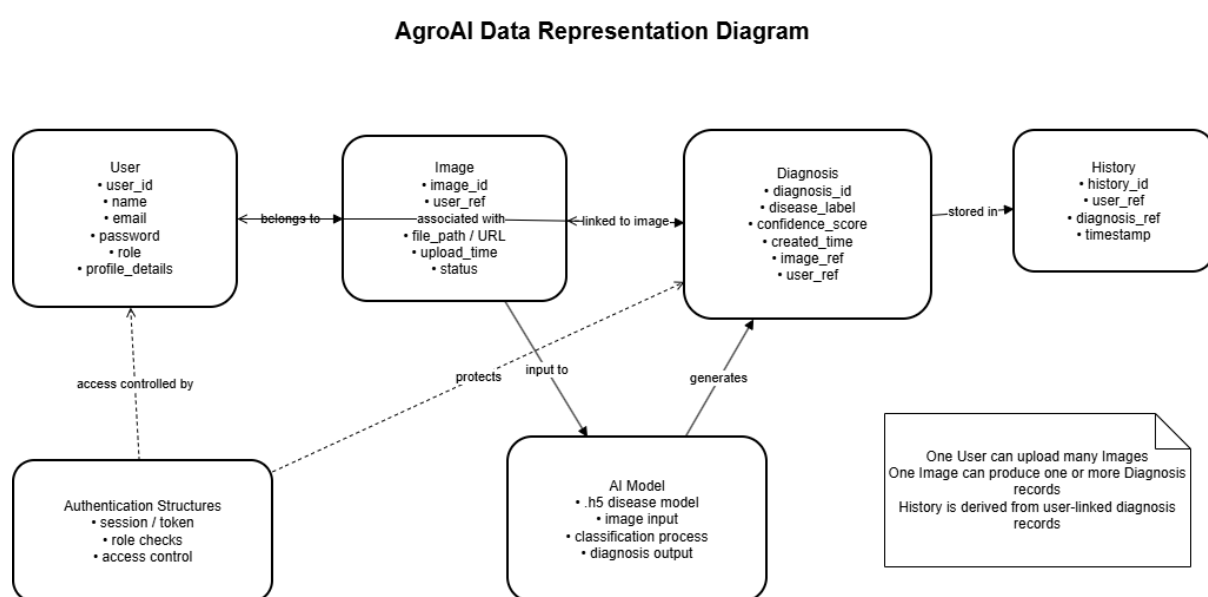


Figure 4-2 Data Representation of AgroAI

Table 4.2.1 Data Representation of AgroAI

Entity	Key Attributes	Description
User	user_id, name, email, password, role, profile_details	Stores user account and authentication related information. It is the main entity because all personalized records and activities are linked to the user.
Image	image_id, user_ref, file_path/URL, upload_time, status	Stores information about the plant leaf image uploaded by the user. It acts as the main input entity for the disease diagnosis process.
Diagnosis	diagnosis_id, disease_label, confidence_score, created_time, image_ref, user_ref	Stores the AI generated disease prediction result. It links the diagnosis to both the uploaded image and the user.
History	history_id, user_ref, diagnosis_ref, timestamp	Maintains previous diagnosis records of the user for later viewing and tracking.
Calendar Record	scan_id, user_ref, diagnosis_ref, scan_date	Stores scan dates so that users can view plant diagnosis activity in calendar form.
Disease Trend	trend_id, user_ref, disease_type, frequency, time_period	Represents disease pattern information generated from stored diagnosis records.
Farm Map	farm_id, user_ref, boundary_coordinates, location_name	Stores saved farm boundary and map related data for user specific farm visualization.

AI Model	model_type, image_input, classification_process, diagnosis_output	Represents the intelligent component that processes uploaded images and generates disease prediction results.
Authentication Structures	session/token, role_checks, access_control	Supports secure login, session handling, and access control for protected system features.

4.3 Process Flow / Representation

Process flow representation explains how AgroAI performs its major functions step by step during actual execution. While the architecture section shows the structural parts of the system, the process flow section explains how these parts cooperate when a user performs an action. This is especially important in AgroAI because the platform includes multiple layers: user interface, backend processing, database interaction, and AI inference. A process flow representation helps connect these structural components into a meaningful operational sequence.

The most important process in AgroAI is the **disease detection flow**. This flow begins when a user interacts with the web or mobile application and selects the disease diagnosis feature. The user uploads a plant leaf image or captures one through the supported interface. At this stage, the frontend collects the input and sends it to the backend service. The backend receives the image and performs the required processing and validation steps. Once the request is ready, the backend forwards the plant image to the AI inference layer, where the deployed machine learning model performs analysis. The AI model examines the visual features of the image and returns a disease prediction together with a confidence value. The backend then receives this output, stores the relevant diagnosis information in the database, and sends the result back to the user interface. Finally, the frontend displays the prediction result to the user in a readable form. This sequence forms the central process flow of the platform because it directly corresponds to the main purpose of AgroAI.

The disease detection flow can be described more clearly through individual stages. The first stage is **input acquisition**, where the user provides a plant leaf image. The second stage is **request handling**, where the backend accepts and prepares the image data. The third stage is

AI inference, where the trained model classifies the image. The fourth stage is result handling, where the backend receives and organizes the prediction output. The fifth stage is record storage, where the diagnosis information is saved in the database. The sixth and final stage is output presentation, where the result is displayed to the user. Structuring the process in this way makes it easier to understand the path taken by the request and highlights the role of each system component in delivering the final diagnosis.

The user interaction flow is equally an important process in the system. This flow begins when the user opens the platform and creates an account or logs in with valid credentials. The frontend submits the login or registration information to the backend. The backend validates the request, communicates with the database, and either creates a new account or authenticates the existing one. Once authentication is successful, the user is granted access to the main system interface. From there, the user moves to the dashboard, uploads images, views diagnosis results, accesses history, and updates profile details. This process shows that AgroAI is not just a technical AI pipeline but a complete application with controlled user access and ongoing interaction support.

A third useful process is the **history retrieval flow**. This begins when the user opens the history section of the platform. The frontend sends a request to the backend asking for stored diagnosis records associated with the authenticated account. The backend validates the identity of the user, retrieves the relevant diagnosis records from the database, and returns them in a structured form. The frontend then displays the stored records to the user. This flow is significant because it illustrates how the platform supports persistent interaction beyond one time disease prediction.

A fourth optional process flow is **profile management**. In this process, the user opens the profile section, reviews current account information, modifies one or more fields, and submits the updated data. The backend validates the changes, updates the relevant record in the database, and returns confirmation to the frontend. Although this process is not as central as disease detection, it remains useful for completing the user centered design of the platform.

For academic presentation, this section includes at least one main figure. The most important figure is the disease detection process, since it represents the core value of AgroAI. A second figure may be added for user interaction or history retrieval if required by the level of detail desired in the report.

AgroAI Process Flow Representation

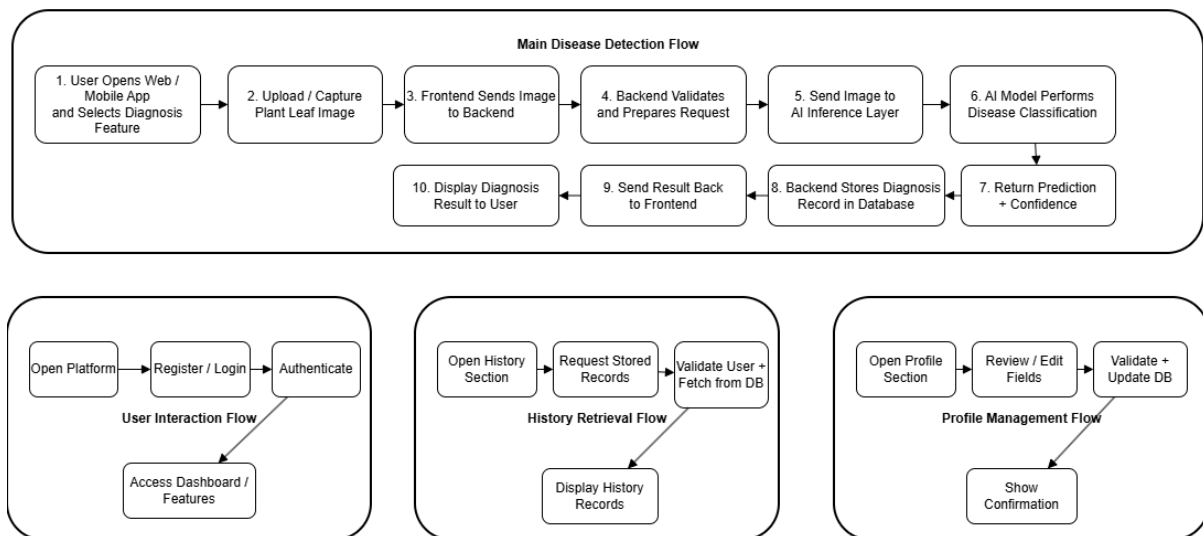


Figure 4-3 Flow Representation of AgroAI

4.4 Design Models

Design models provide a more detailed and formal representation of the internal structure and behavior of AgroAI. They help convert the functional requirements identified earlier into visible design structures that explain how the system behaves and how its entities are related. In AgroAI, the most important design models are the **sequence diagram** and the **class diagram**. Together, these models describe both the dynamic interaction of the system and its static structural organization.

4.4.1 Sequence Diagram

A sequence diagram shows how system components communicate over time during execution of a particular function. In AgroAI, the most important sequence diagram is the one representing the disease diagnosis workflow, since this is the core function of the project. The sequence begins with the **User**, who interacts with the **Frontend** by uploading a plant leaf image. The frontend then sends the image to the **Backend**. The backend forwards the data to the **AI Model** for analysis. Once the AI model returns the predicted disease and confidence value, the backend stores the result in the **Database** and sends the final response back to the frontend. The frontend finally displays the diagnosis result to the user.

This sequence model is important because it makes the chronological order of operations explicit. It shows that the system does not work through isolated modules operating

independently; rather, the major components cooperate in a fixed order to complete the diagnosis task. The sequence diagram also shows the central role of the backend, since all communication between user interface, AI model, and database passes through it. This reflects the client server nature of the platform and demonstrates how system coordination is performed in practice.

In addition to disease detection, the sequence diagram may also represent the login process. In that case, the user submits credentials through the frontend, the backend verifies those credentials with the database, and successful authentication leads to access being granted. Whether the sequence diagram includes one or both flows, its main role is to demonstrate the ordered exchange of information among the major components of AgroAI.

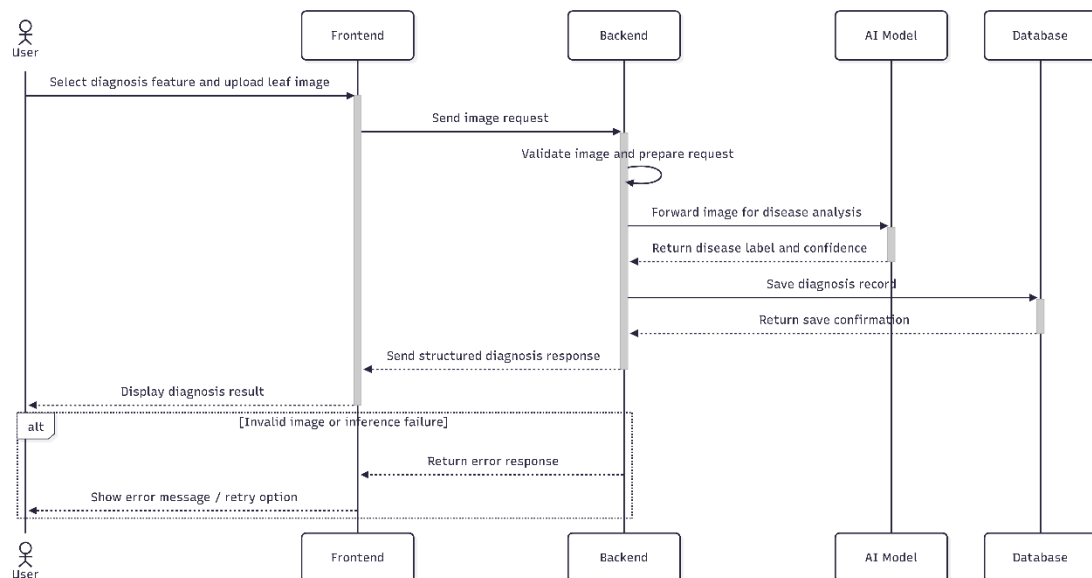


Figure 4-4 Sequence Diagram of AgroAI

4.4.2 Class Diagram

A class diagram represents the structural design of the platform by showing the major classes or conceptual entities and their relationships. In AgroAI, the class diagram focuses on the most important system entities: User, Image, Diagnosis, AI Model, and History. These classes capture the essential informational and functional structure of the project.

The User class represents registered users of the system. It includes attributes related to identity, authentication, and profile information. The Image class represents uploaded plant leaf images

and includes attributes such as file reference, validation state, and upload time. The Diagnosis class represents the result of disease prediction and includes the predicted disease label, confidence score, and time of diagnosis. The AI Model class represents the intelligent component responsible for classification. The History class represents the stored sequence of diagnosis results associated with a user. These classes are related in a meaningful way: one user owns multiple images and multiple diagnosis records, and one diagnosis record is associated with a specific image and a specific user.

The class diagram is useful because it gives a clear static picture of how AgroAI is organized conceptually. Even though the final system is implemented through modern frameworks and service based logic rather than a strictly traditional object oriented desktop structure, the class diagram remains valuable in academic documentation. It helps explain the main entities of the platform and demonstrates how requirements are transformed into an organized design.

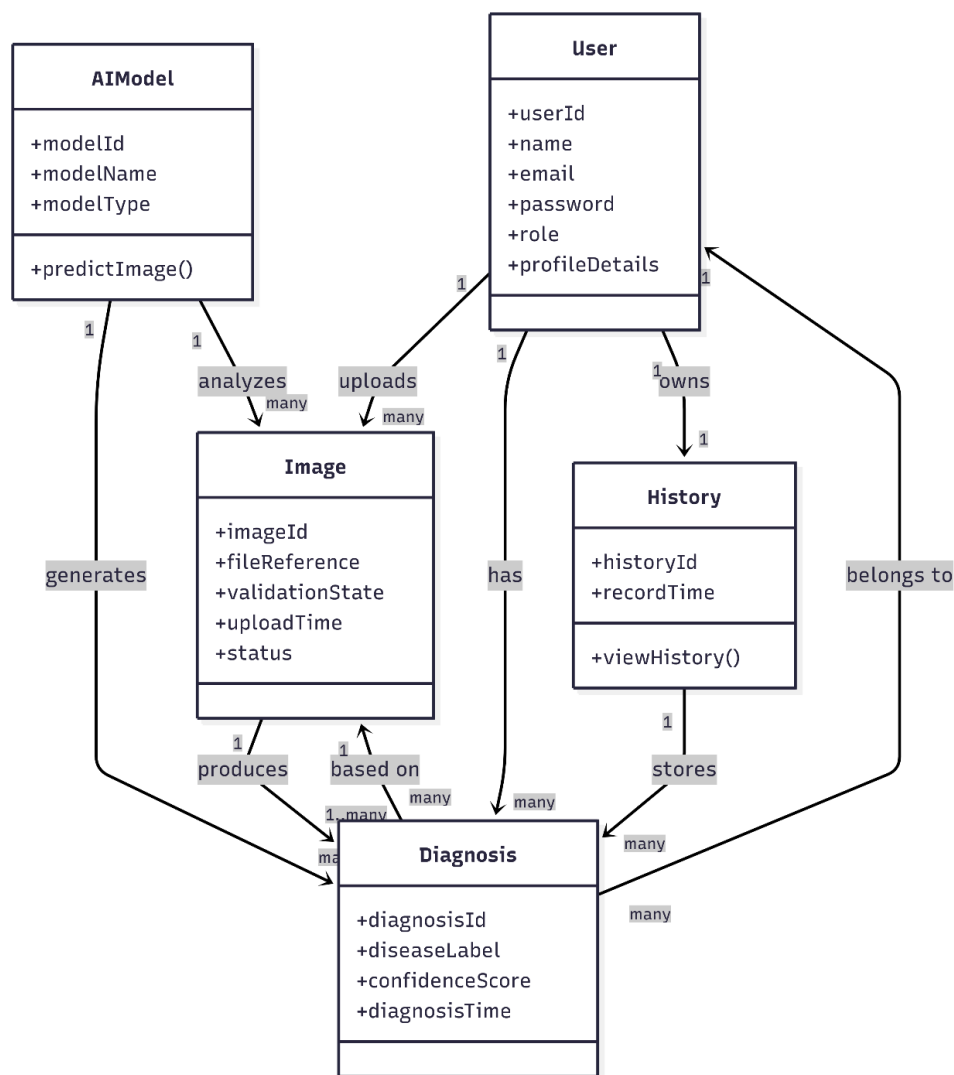


Figure 4-5 Class Diagram of AgroAI

5 Implementation

5.1 Algorithm

The implementation of AgroAI focuses on converting the system design into a working software platform that performs plant disease diagnosis through an integrated web, mobile, backend, database, and AI environment. This chapter explains how the main functions of the system are implemented in practice. In accordance with the project report template, this chapter includes the algorithmic logic of the system, the external services or APIs used during implementation, and the user interface through which users interact with the platform. The chapter does not include full source code; instead, it explains the major implementation logic and working behavior of the system in a structured way.

The core implementation of AgroAI centers on the plant disease detection workflow. This workflow begins from user input and continues until a diagnosis result is displayed and stored. Since the platform is implemented as a cross platform system, the same functional logic supports both the web and mobile interfaces. The frontend collects user input, the backend performs request handling and communication, the AI inference service processes the plant leaf image, and the database stores the result for future use. This flow turns the conceptual design of the project into a practical software process.

The intelligent component that powers the disease detection workflow is AgroNet, a custom Convolutional Neural Network (CNN) designed and built using the Keras Sequential API, stored in .h5 format. AgroNet follows a deep sequential convolutional architecture inspired by the VGGNet design principle of stacking pairs of Conv2D layers before each pooling operation, but is trained entirely from scratch on the PlantVillage dataset rather than relying on pre-trained weights from architectures such as VGG16, ResNet50, or MobileNetV2.

The internal architecture of the model follows a well established CNN design pattern in which convolutional blocks are stacked progressively to extract increasingly complex visual features from the input image. The model contains five such convolutional blocks. Each block consists of two Conv2D layers followed by a MaxPooling2D layer. In the first block, the Conv2D layers use 32 filters. In the second block, the filter count increases to 64. The third block uses 128 filters, the fourth uses 256, and the fifth uses 512. This progressive doubling of filter depth allows the model to learn low level features such as edges and textures in the earlier layers, and higher level disease related visual patterns such as lesion shapes, discoloration regions, and

spot distributions in the deeper layers. The MaxPooling2D layers reduce the spatial dimensions of the feature maps after each block, which controls computational cost and helps the model focus on the most prominent features.

After the five convolutional blocks, a Dropout layer is applied to reduce overfitting by randomly deactivating a portion of neurons during training. The feature maps are then passed through a Flatten layer, which converts the multi dimensional output into a one dimensional vector suitable for fully connected processing. This vector is passed into a Dense layer containing 1500 neurons, followed by a second Dropout layer for additional regularization. The final output layer is a Dense layer with 38 neurons and a softmax activation function, which produces a probability distribution across all 38 disease classes. The class with the highest probability is returned as the predicted disease label, and its associated probability value is used as the confidence score presented to the user. In total, the model contains approximately 7.84 million trainable parameters, which makes it sufficiently expressive for multi class plant disease classification while remaining deployable within the constraints of the Hugging Face Spaces inference environment.

The main disease detection logic of AgroAI can be explained through the following algorithmic sequence:

1. The user opens the disease detection feature through the web or mobile interface.
2. The system allows the user to upload or capture a plant leaf image.
3. The frontend validates the file type and prepares the image for submission.
4. The image is sent to the backend service.
5. The backend receives the image and forwards it to the AI inference service.
6. The AI inference service resizes the image to 128×128 pixels and normalizes the pixel values before passing it through the CNN model.
7. The model processes the image through its five convolutional blocks and fully connected layers, producing a probability score for each of the 38 disease classes.
8. The AI service returns the predicted disease label and confidence score to the backend.
9. The backend stores the diagnosis result in the database with a reference to the authenticated user.

10. The backend sends the structured result to the frontend.

11. The frontend displays the disease diagnosis result to the user.

From the user's perspective, this entire process takes a few seconds they upload a photo and see a result. What is less visible is how much coordination happens between the components at that time. The frontend validates the image format before anything is sent to the backend. The backend does not perform the classification itself; it acts as a relay, passing the image to the inference service and waiting for a structured response before storing the record and returning data to the frontend. Each step depends on the previous one completing correctly, which is why integration testing between these layers was treated as a priority during development.

5.1.1 AgroNet CNN Architecture and Working

The plant disease detection model used in AgroAI is named AgroNet, a custom Sequential CNN built with the Keras Sequential API and trained entirely from scratch on the PlantVillage dataset. The architecture follows a VGGNet-inspired design principle of stacking pairs of Conv2D layers before each pooling operation, with five progressive convolutional blocks of increasing filter depth.

Table 5.1.1 AgroNet Layer-by-Layer Architecture

Layer	Type	Configuration	Output Shape	Parameters
Input	Input	128 × 128 × 3 RGB Image	128 × 128 × 3	—
conv2d	Conv2D	32 filters, 3 × 3, padding='same', ReLU	128 × 128 × 32	896
conv2d_1	Conv2D	32 filters, 3 × 3, padding='valid', ReLU	126 × 126 × 32	9,248
max_pooling2d	MaxPooling2D	Pool size = 2 × 2	63 × 63 × 32	0
conv2d_2	Conv2D	64 filters, 3 × 3, padding='same', ReLU	63 × 63 × 64	18,496

conv2d_3	Conv2D	64 filters, 3×3 , padding='valid', ReLU	$61 \times 61 \times 64$	36,928
max_pooling2d_1	MaxPooling2D	Pool size = 2×2	$30 \times 30 \times 64$	0
conv2d_4	Conv2D	128 filters, 3×3 , padding='same', ReLU	$30 \times 30 \times 128$	73,856
conv2d_5	Conv2D	128 filters, 3×3 , padding='valid', ReLU	$28 \times 28 \times 128$	147,584
max_pooling2d_2	MaxPooling2D	Pool size = 2×2	$14 \times 14 \times 128$	0
conv2d_6	Conv2D	256 filters, 3×3 , padding='same', ReLU	$14 \times 14 \times 256$	295,168
conv2d_7	Conv2D	256 filters, 3×3 , padding='valid', ReLU	$12 \times 12 \times 256$	590,080
max_pooling2d_3	MaxPooling2D	Pool size = 2×2	$6 \times 6 \times 256$	0
conv2d_8	Conv2D	512 filters, 3×3 , padding='same', ReLU	$6 \times 6 \times 512$	1,180,160
conv2d_9	Conv2D	512 filters, 3×3 , padding='valid', ReLU	$4 \times 4 \times 512$	2,359,808
max_pooling2d_4	MaxPooling2D	Pool size = 2×2	$2 \times 2 \times 512$	0
dropout	Dropout	Rate = 0.25	$2 \times 2 \times 512$	0
flatten	Flatten	Converts feature maps into a 1D vector	2048	0
dense	Dense	1500 neurons, ReLU	1500	3,073,500
dropout_1	Dropout	Rate = 0.40	1500	0
dense_1 (Output)	Dense	38 neurons, Softmax	38	57,038
Total Parameters				7,842,762

How AgroNet Processes an Image:

When a plant leaf image is submitted, AgroNet processes it through the following stages:

In Blocks 1 and 2 (32 and 64 filters), the early convolutional layers detect low-level visual features such as color gradients, edges, and surface textures. The alternating use of padding='same' in the first Conv2D of each block and no padding in the second controls the spatial dimension reduction within each block.

In Blocks 3 and 4 (128 and 256 filters), the model begins recognizing intermediate disease patterns. With increased filter depth, these layers capture more complex structures such as irregular spot shapes, discolored leaf regions, blight boundaries, and fungal surface patterns that are specific to certain disease categories.

In Block 5 (512 filters), the spatial resolution has been reduced significantly through successive pooling. With the deepest filter count, this block extracts the highest-level disease representations that differentiate one class from another, such as rust spots, powdery mildew texture, and lesion coverage patterns.

After Block 5, Dropout (rate 0.25) is applied to prevent overfitting, followed by a Flatten layer that converts the feature maps into a single vector of 2048 values. This passes through a Dense layer of 1500 neurons with ReLU activation for high-capacity classification over the extracted features. A second Dropout layer (rate 0.4) is applied for further regularization before the final output layer, which uses Softmax activation across 38 neurons — one per disease class. The neuron with the highest probability is the predicted disease, and that probability value is presented to the user as the confidence score.

5.1.2 Image Upload and Validation Process

The image upload and validation process is an important part of the AgroAI implementation because the quality and format of the input image directly affect the prediction result. Since the system depends on image based plant disease diagnosis, the uploaded image must be handled carefully before it is forwarded to the AI inference service. The purpose of validation is to

ensure that the system receives a proper image file and rejects unsupported or incorrect input before processing.

When the user opens the scan or disease detection screen, the frontend provides an option to upload or capture a plant leaf image. In the web application, the user can select an image file from the device storage. In the mobile application, the user may either select an existing image or capture a new image using the device camera. After the image is selected, the frontend performs initial validation before sending the image to the backend.

The frontend validation may include checking whether the file is selected, whether the file format is supported, and whether the file appears to be an image. This step prevents unnecessary or invalid requests from reaching the backend. For example, if the user selects a document, video, empty file, or unsupported file type, the system should not forward it to the AI model. Instead, it should show a clear message asking the user to upload a valid plant leaf image.

The backend also performs validation after receiving the image. This second level of validation is important because frontend validation alone is not enough for a reliable system. A user may bypass frontend restrictions or send an invalid request directly to the backend. Therefore, the backend must confirm that the received file is valid before forwarding it to the AI inference service. This improves the security, reliability, and stability of the system.

The image upload and validation process can be summarized in the following:

1. User selects or captures a plant leaf image
2. Frontend checks whether the file is selected
3. Frontend verifies that the file is an image
4. Invalid files are rejected with an error message
5. Valid image is sent to the backend
6. Backend performs additional validation
7. Backend prepares the image for inference
8. Image is forwarded to the AI inference service

This validation process improves the quality of system input and prevents unnecessary processing of invalid files. It also improves user experience because the user receives immediate feedback when an incorrect file is selected.

5.1.3 AI Model Inference Workflow

The AI model inference workflow is the central intelligent process of AgroAI. It begins after the backend receives a valid plant leaf image and forwards it to the deployed AI service. The AI model is stored in .h5 format and deployed through Hugging Face Spaces. This deployment allows AgroAI to perform live prediction instead of keeping the model only in an offline testing environment.

During inference, the uploaded image is converted into the same input format that was used during model training. This is necessary because a machine learning model produces reliable results only when the input format during prediction is consistent with the format used during training. The image is resized to 128×128 pixels and represented using three RGB color channels. The pixel values are then normalized so that the image data becomes suitable for numerical processing by the CNN model.

After preprocessing, the image passes through the convolutional layers of the CNN. The earlier layers identify simple features such as edges, curves, color differences, and small texture changes. The deeper layers identify more complex plant disease features such as spots, blight regions, fungal marks, damaged areas, yellowing, and irregular leaf patterns. These extracted features are then passed to the fully connected layers for classification.

The final output layer produces probability values for all 38 supported disease classes. The class with the highest probability is selected as the final predicted disease. The associated probability value is used as the confidence score. For example, if the model assigns the highest probability to a specific disease class, that class is returned as the prediction. If the confidence score is high, the result is considered more reliable. If the confidence score is low, the result may still be displayed, but it indicates that the model is less certain about the classification.

The inference workflow can be summarized as follows:

Table 5.1.2 AI Model Interface Workflow

Stage	Operation
Image Input	Valid plant leaf image is received
Preprocessing	Image is resized to 128×128 pixels and normalized
Feature Extraction	CNN layers extract visual disease patterns
Classification	Dense layers calculate class probabilities
Prediction Selection	Highest probability class is selected
Result Output	Disease label and confidence score are returned

This workflow allows AgroAI to perform automated plant disease diagnosis in a practical way. The user only sees the final disease prediction, but internally the system performs image preprocessing, feature extraction, classification, and result generation.

5.1.4 Backend Coordination Logic

The backend plays a central role in the implementation of AgroAI because it connects the frontend, database, and AI inference service. It acts as the main coordination layer of the platform. The frontend does not directly communicate with the database or the AI model. Instead, all important requests pass through the backend, which validates, processes, forwards, and stores data where required.

When a user uploads a plant leaf image, the backend first receives the request from the web or mobile application. It checks whether the request is valid and whether the user is authenticated. If the user is not logged in, the backend prevents access to protected features. If the user is authenticated, the backend continues processing the request.

For disease prediction, the backend forwards the uploaded image to the AI inference service. It waits for the AI service to return the predicted disease label and confidence score. Once the prediction result is received, the backend organizes the response into a structured format. This structured response is then stored in MongoDB as a diagnosis record and also returned to the frontend for display.

The backend also controls history retrieval, profile access, and user specific data handling. When the user opens the diagnosis history page, the frontend sends a request to the backend. The backend verifies the identity of the logged in user and retrieves only those diagnosis records that belong to that specific user. This ensures that one user cannot access the private records of another user.

The backend coordination logic improves the security and maintainability of AgroAI. Since all important operations pass through the backend, the system becomes easier to control and update. If the AI model endpoint changes in the future, only the backend communication logic needs to be updated. The frontend can remain mostly unchanged because it continues to communicate with the same backend service.

5.1.5 Authentication Algorithm

User authentication is another major implemented function of AgroAI. Authentication is necessary because the system stores user specific diagnosis records, scan dates, profile information, and history. Without authentication, the system would not be able to separate one user's records from another user's records. Therefore, the authentication algorithm ensures that only registered and verified users can access protected features.

The authentication process begins when a user registers or logs in to the system. During registration, the user provides required details such as name, email, and password. The frontend sends this information to the backend. The backend validates the data and checks whether the email is already registered. If the email is unique and all required information is valid, the backend creates a new user account and stores it in the database.

During login, the user provides email and password. The frontend sends these credentials to the backend. The backend verifies whether the email exists and whether the password is correct. If the credentials are valid, the user is authenticated and allowed to access protected modules such as image upload, diagnosis history, calendar, trends, farm map, and profile management.

The authentication flow is as follows:

1. User opens the registration or login screen.
2. User enters the required account information.
3. Frontend submits the information to the backend.

4. Backend validates the received data.
5. For registration, the backend checks whether the email is already used.
6. If the email is unique, a new account is created.
7. For login, the backend verifies the user credentials.
8. If credentials are valid, the user is authenticated.
9. Authenticated user is allowed to access protected features.
10. Invalid credentials result in an error message.

This authentication algorithm supports secure and personalized system interaction. It also ensures that diagnosis records, profile data, and user history remain connected to the correct account.

5.1.6 Diagnosis Record Storage Algorithm

Diagnosis record storage is an important part of AgroAI because the system is not designed only for one time disease prediction. It also allows users to view previous scans, monitor disease history, observe disease trends, and check scan dates in the calendar. For this reason, each diagnosis result must be stored properly in the database.

After the AI model returns a prediction, the backend stores the diagnosis record in MongoDB. The stored record may include user reference, image reference, predicted disease name, confidence score, scan date, and result status. This information allows the system to display the diagnosis history later.

A typical diagnosis record may include the following fields:

Table 5.1.2 Diagnosis Record Storage Algorithm

Field	Purpose
User ID	Connects the diagnosis to the correct user
Image Reference	Stores the path or URL of the uploaded image
Disease Label	Stores the predicted disease name
Confidence Score	Stores the model certainty value

Scan Date	Stores the date and time of diagnosis
Status	Shows whether prediction was completed successfully

This storage structure is useful because it keeps every prediction connected to the correct user and image. Without this relationship, the system would not be able to display personalized history, scan records, disease trends, or calendar based scan dates.

The diagnosis storage algorithm is as follows:

1. Backend receives prediction result from the AI inference service.
2. Backend extracts predicted disease label and confidence score.
3. Backend connects the result with the authenticated user.
4. Backend stores image reference and prediction result in MongoDB.
5. Stored record becomes available for history, trends, and calendar features.
6. Backend sends response to frontend for display.

This storage process gives AgroAI practical value beyond simple prediction. It allows the system to maintain a record of user activity and supports future monitoring features.

5.1.7 Diagnosis History Retrieval Algorithm

The diagnosis history feature allows users to view their previous disease diagnosis records. This feature is important because plant disease monitoring is usually not a one time activity. A user may need to review previous results, compare disease occurrences, or check when a specific plant was scanned. Therefore, the history retrieval algorithm makes AgroAI more useful for repeated crop health monitoring.

The history retrieval process begins when the authenticated user opens the history section. The frontend sends a request to the backend to fetch previous records. The backend verifies the user identity and then queries MongoDB for diagnosis records linked to that user. The retrieved records are sent back to the frontend, where they are displayed in a structured list or history view.

The diagnosis history retrieval algorithm is as follows:

1. Authenticated user opens the diagnosis history section.
2. Frontend sends a history request to the backend.
3. Backend verifies the identity of the logged in user.
4. Backend queries MongoDB for records linked to that user.
5. Database returns the relevant diagnosis records.
6. Backend sends the records to the frontend.
7. Frontend displays the history in a structured format.

This algorithm ensures that each user can access only their own records. It also supports better organization of diagnosis information because previous results are stored and available for later review.

5.1.8 Disease Trends and Calendar Data Logic

AgroAI also includes features such as disease trends and calendar based scan records. These features are connected to the diagnosis record storage algorithm because they depend on previously saved scan data. Once diagnosis records are stored in the database, the system can reuse that data to provide more meaningful views to the user.

The disease trends feature analyzes stored diagnosis records and presents them in a summarized form. For example, the system may show which disease appears most frequently in the user's scan history or how disease occurrences change over time. This helps users observe repeated disease patterns and understand the general condition of their crops.

The calendar feature uses the scan date stored with each diagnosis record. When the user opens the calendar section, the system retrieves diagnosis records and matches them with their saved dates. This allows the user to see on which dates plant scans were performed. The calendar view makes the diagnosis history easier to understand because records are arranged according to time.

The data logic for trends and calendar features can be summarized as follows:

Table 5.1.3 Disease Trends and Calendar Data Logic

Feature	Data Used	Purpose
Disease Trends	Disease labels and scan records	Shows repeated disease patterns
Calendar View	Scan dates and diagnosis records	Shows dates on which scans were performed
History View	Complete diagnosis records	Allows review of previous results
Farm Map	Saved farm boundary data	Shows farm area connected with user records

These features increase the practical value of AgroAI because they make the system more than a simple prediction tool. They allow the user to monitor crop health activity over time and review previous results in an organized way.

5.1.9 Error Handling in Algorithmic Workflow

Error handling is necessary in AgroAI because the system depends on multiple connected components. The frontend, backend, database, and AI inference service must all work correctly for the complete workflow to succeed. If one part fails, the system should handle the issue properly instead of crashing or showing confusing output.

Several errors may occur during the disease diagnosis process. A user may upload an invalid file, the image may fail to reach the backend, the AI inference service may become temporarily unavailable, or the database may fail to store the result. Each situation requires a proper system response.

For invalid image uploads, the frontend should show a message asking the user to upload a valid plant leaf image. For authentication errors, the system should restrict access and ask the user to log in. If the AI service is unavailable, the backend should return a clear failure message instead of leaving the user waiting. If database storage fails, the system may still display the prediction result but should indicate that the record could not be saved.

The following table summarizes possible errors and system responses:

Table 5.1.4 Error Handling in Algorithmic Workflow

Error Situation	System Response
Empty image upload	Ask user to select an image
Unsupported file type	Show invalid file format message
User not logged in	Restrict access and request login
Backend connection failure	Show request failure message
AI service unavailable	Show prediction service unavailable message
Low confidence prediction	Display result with caution
Database save failure	Show result but notify that record was not saved
No history available	Display empty history message

Error handling improves the reliability of AgroAI because it allows the system to respond properly under unexpected conditions. It also improves user experience because users receive understandable messages instead of technical errors.

5.2 External APIs

The implementation of AgroAI depends not only on internal application logic but also on the use of external services and APIs that support communication, inference, and structured data exchange. In modern full stack systems, external APIs often play an important role because they allow one part of the platform to request services from another part in a standardized way. In AgroAI, APIs are particularly important for connecting the frontend with the backend and for connecting the backend with the AI inference service. This makes the API layer an essential part of implementation. The project report template also specifically requires a section describing the APIs used in the project.

The most important external service used in AgroAI is the **AI inference service** hosted through **Hugging Face Spaces**. The trained plant disease classification model, stored in .h5 format, is

deployed there for live inference. When the user uploads a plant leaf image, the backend communicates with this inference service by sending the image data and receiving the prediction response. This service acts as the AI processing endpoint of the system. It returns important information such as the predicted disease label and the associated confidence value, which are then used by the backend to generate the result shown to the user.

In implementation terms, the AI inference service works like a specialized prediction API. The backend does not perform the disease classification locally inside the same layer that handles authentication and record management. Instead, it sends the image to the inference layer, waits for the result, and then continues its normal application flow. This approach keeps the backend logic cleaner and allows the model to remain separately deployable. It also supports better modularity, because changes in the AI model can be managed more independently from the rest of the system.

The backend of AgroAI itself exposes internal API endpoints for both the web and mobile applications. These endpoints support the major functions of the platform, including:

- user registration
- user login
- image upload for disease detection
- diagnosis history retrieval
- profile management

The web application and mobile application both communicate with the same backend service through these API routes. This is an important aspect of implementation because it ensures consistency across platforms. The same backend logic supports both clients, which reduces duplication of system functionality and keeps the project architecture organized.

A typical backend API interaction for plant disease detection works as follows:

1. The frontend sends the uploaded image to the backend endpoint.
2. The backend receives and validates the request.
3. The backend forwards the image to the AI inference service.
4. The prediction response is returned to the backend.

5. The backend stores the result and sends the final structured response to the frontend.

This API driven approach makes the system more maintainable and scalable, because each request follows a defined path and the responsibilities of each service remain clear.

Table 5.2.1 External APIs of AgroAI

Name of API / Service	Description	Purpose of Usage	Used In
Backend Authentication API	Handles registration and login requests	User account access and validation	Authentication module
Disease Detection API	Accepts uploaded image and returns diagnosis result	Core plant disease prediction workflow	Detection module
History Retrieval API	Fetches diagnosis history of authenticated user	Record viewing and monitoring	History module
Profile Management API	Updates and retrieves user profile information	User account maintenance	Profile module
Hugging Face Inference Service	Processes image using deployed .h5 model	AI based plant disease classification	Prediction service

5.3 User Interface

The user interface of AgroAI serves as the main point of interaction between the user and the system. It is designed to provide a simple, clear, and accessible environment through which users can perform all major functions of the platform. Since AgroAI is implemented for both web and mobile environments, the interface is developed with the objective of maintaining consistency across platforms while ensuring ease of use. The mobile interface is developed using Flutter, whereas the web interface is implemented using the MERN stack with TypeScript, with React and TypeScript used for frontend development. Despite the difference in implementation technologies, both interfaces provide the same core features and support a unified user experience.

The registration interface allows a new user to create an account in the system. It includes the necessary input fields for user information and applies validation rules to ensure that only correct and complete data is submitted. This screen is important because it provides the first

formal interaction between the user and the platform. A clear registration interface improves accessibility and supports smooth onboarding of new users.

The login interface enables registered users to access the system using valid credentials. Since several important features of AgroAI, such as diagnosis history and profile management, require authenticated access, the login screen acts as the entry point to the personalized functions of the platform. Its implementation focuses on clarity, proper error handling, and smooth redirection to the dashboard after successful authentication.

The dashboard interface acts as the main navigation screen of the platform after login. It presents the user with organized access to the major modules of the system, including disease detection, diagnosis history, calendar based scan records, farm boundary monitoring, disease trend observation, and profile management. The dashboard is important because it provides a structured overview of the system and helps users move easily between different platform features.

The leaf image upload interface is one of the most important screens in AgroAI because it supports the core functionality of the platform. Through this screen, the user uploads or captures a plant leaf image for disease analysis. The design of this interface focuses on simplicity and directness so that the image submission process remains easy to understand and execute. Since the disease diagnosis process begins from this screen, its usability directly affects the overall effectiveness of the system.

The diagnosis result interface displays the disease prediction returned by the AI model. It presents the predicted disease label and confidence value in a clear and structured manner so that the result can be easily understood by the user. This screen is significant because it transforms the technical output of the AI model into practical information that supports the user in understanding plant health conditions.

The diagnosis history interface allows users to review previously stored diagnosis records. This feature increases the practical value of the platform by supporting long term access to past results rather than limiting the system to one time prediction only. The history screen is implemented in a structured way so that users can view their records in an organized and readable format.

The calendar interface provides a date based representation of scan activity. It highlights the dates on which plant scans are available, thereby allowing users to review diagnosis records

through a calendar view rather than only through a conventional list. This improves the organization of historical data and gives users a clearer understanding of when particular scans are performed.

The farm boundary map interface displays the saved farm boundaries of the user within a map based view. This interface provides a visual representation of agricultural field areas and supports location based review of stored farm information. By showing saved field boundaries directly on the map, the system improves the practicality of visual farm monitoring and extends the platform beyond text based record management.

The disease trends interface presents disease related trends and monitoring information in an organized visual format. This interface strengthens the analytical aspect of the platform by allowing users to observe broader disease related patterns rather than focusing only on individual diagnosis results. Through this screen, AgroAI supports not only detection but also disease awareness and monitoring.

The profile interface supports account management and personalization. It allows the user to view and update account related information in a structured manner. Although it is not the primary analytical component of the platform, it is important for maintaining correct user data and supporting a complete user centered experience.

Overall, the user interface of AgroAI is designed according to the principles of simplicity, accessibility, and structured navigation. All major functions of the platform are made reachable through clear interface elements, and the presentation of information remains organized across both web and mobile environments. This ensures that the technical capabilities of the system are translated into a practical and user friendly software platform.

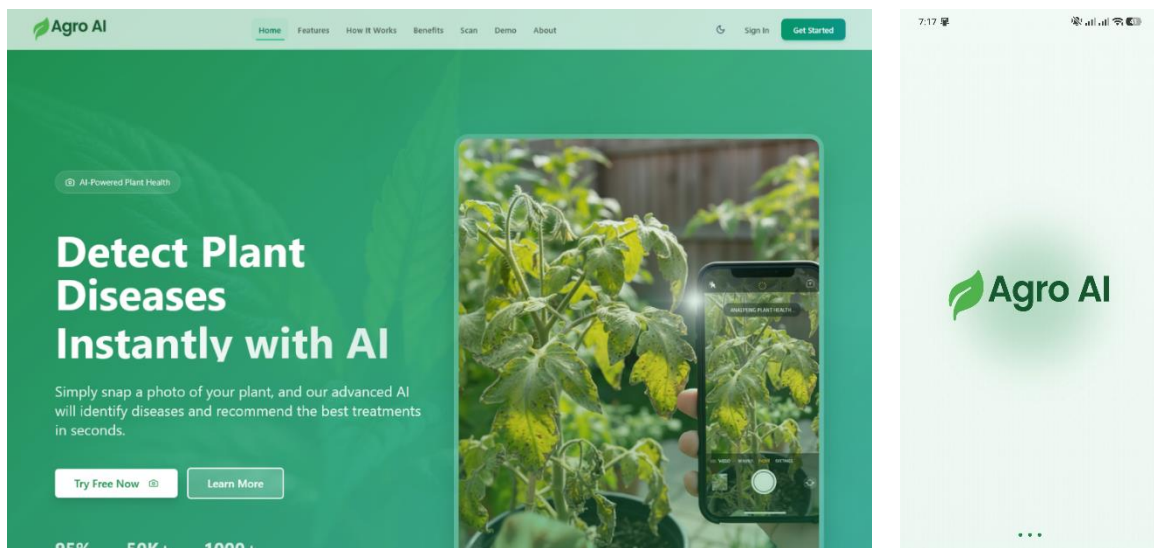


Figure 5-1 Homepage of AgroAI

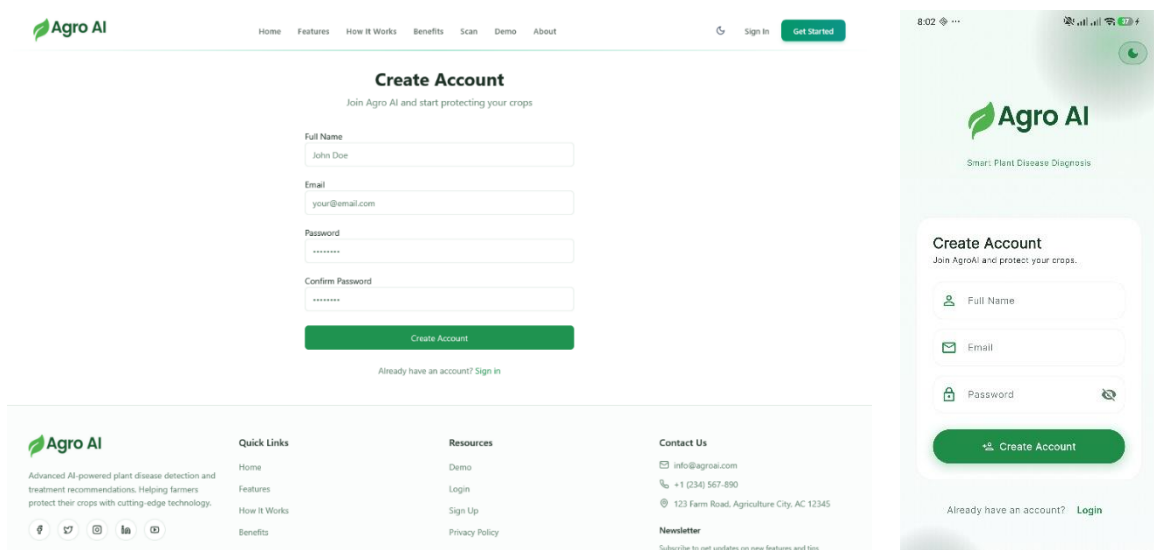


Figure 5-2 Registration of AgroAI

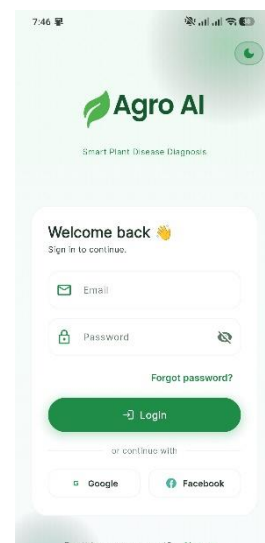
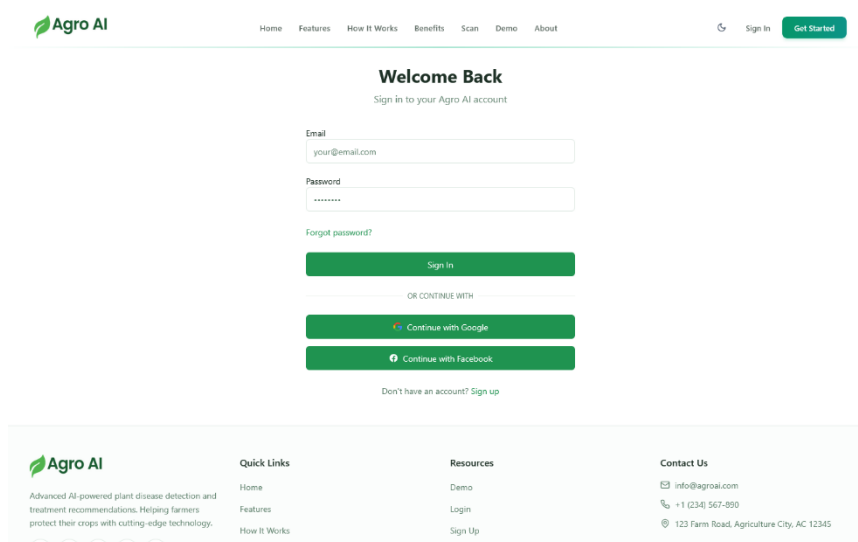


Figure 5-3 Sign In page of AgroAI

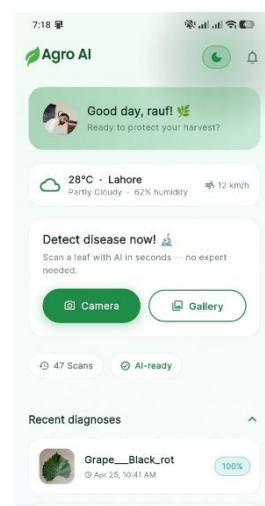
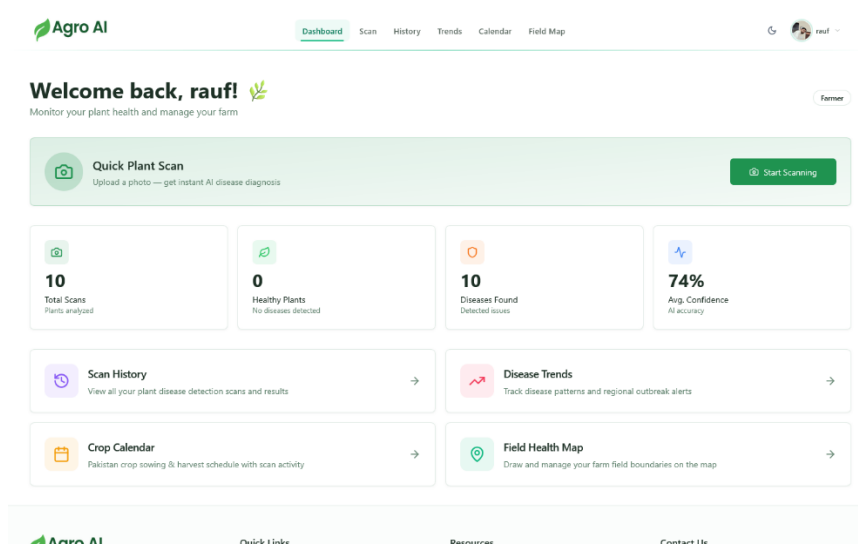


Figure 5-4 Dashboard of AgroAI

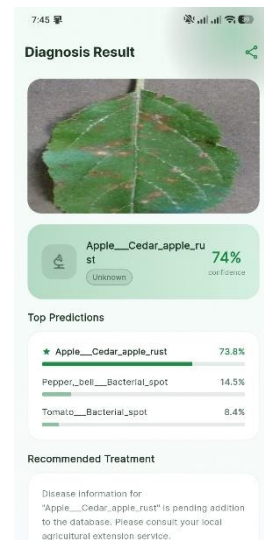
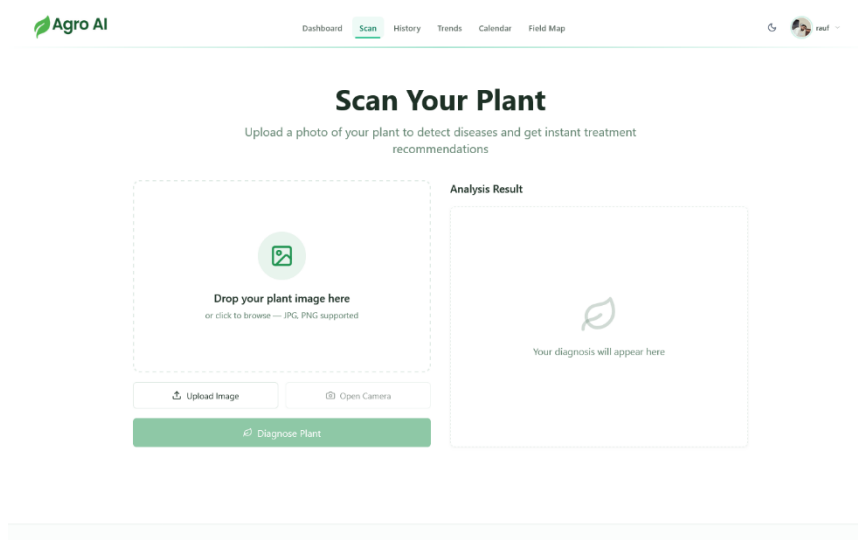


Figure 5-5 Scan page of AgroAI

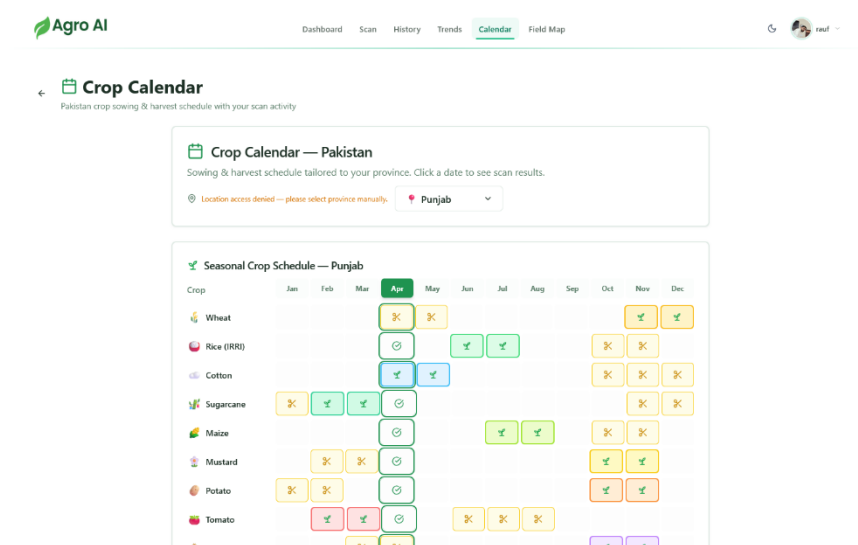


Figure 5-6 Crop Calendar of AgroAI

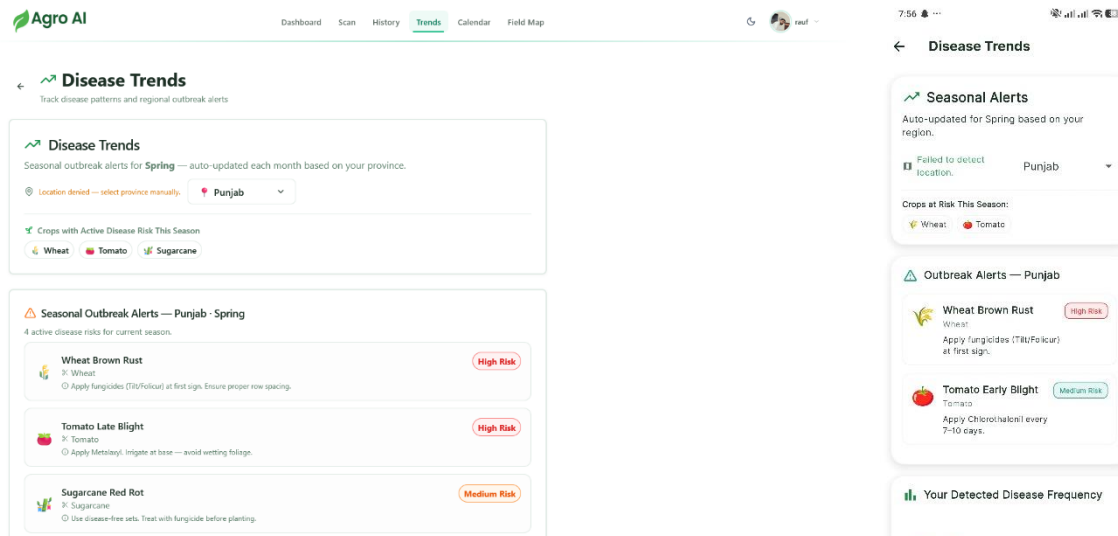


Figure 5-7 Disease Trends of AgroAI

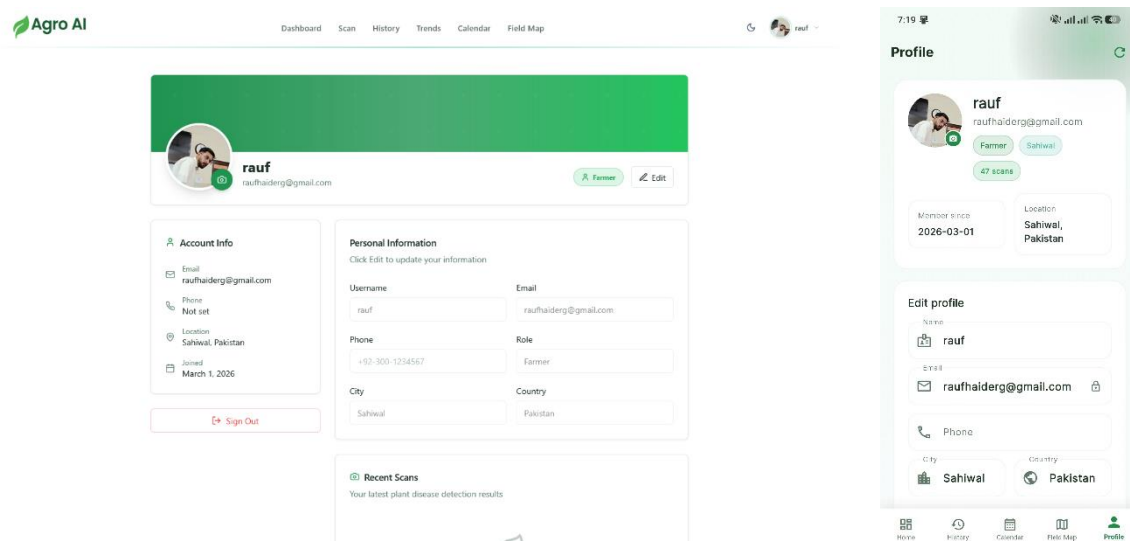


Figure 5-8 Profile Page of AgroAI

6 Testing and Evaluation

6.1 Manual Testing

Testing and evaluation are essential parts of software development because they verify whether the implemented system performs according to the intended requirements. In AgroAI, testing ensures that the platform functions correctly across its major modules, including authentication, image upload, disease prediction, history retrieval, calendar based scan display, farm boundary

map interaction, disease trends viewing, and profile management. Since AgroAI is a cross platform system that combines frontend interfaces, backend services, database storage, and AI based inference, testing is necessary not only to identify technical errors but also to confirm that the complete user experience remains reliable and usable. The report template requires this chapter to include manual testing, system testing, unit testing, functional testing, integration testing, and automated testing.

Manual testing is performed by interacting with the platform directly and observing whether each feature behaves as expected. This form of testing is important because it reflects the actual user experience of the system. In AgroAI, manual testing focuses on the correctness of visible operations such as account registration, login, image submission, result display, record viewing, and navigation across system pages. Since the project includes web and mobile access, manual testing also helps confirm that the implemented features remain consistent across the supported interfaces.

The manual testing process begins with the authentication module. Testers attempt to register new users with valid and invalid information, log in with correct and incorrect credentials, and verify whether access control behaves properly. This confirms that the system accepts valid requests, rejects invalid inputs, and protects restricted features. Manual testing also includes navigation to the dashboard after successful login and access denial when authentication is missing or incorrect.

After authentication, manual testing focuses on the disease detection workflow, which is the core function of AgroAI. In this process, testers upload valid plant leaf images, attempt invalid uploads, and observe whether the system correctly forwards the image for analysis and returns a disease result. The result screen is then checked to verify whether the prediction label and confidence value are shown in a clear and understandable form. This confirms that the image upload flow, backend processing, AI inference, and result presentation all work together correctly.

Manual testing also includes the diagnosis history module. Testers verify whether previous diagnosis records appear in the history view for the authenticated user and whether those records remain hidden from other accounts. The calendar interface is tested by checking whether dates with scan activity are displayed correctly. The farm boundary map interface is tested by confirming that saved boundaries appear in the proper map based view. Similarly, the disease trends page is tested by opening the interface and verifying whether the relevant

monitoring information is displayed. Profile management is also tested by modifying account details and verifying whether updates are stored and shown correctly.

Because AgroAI is a user centered platform, manual testing is not limited to technical correctness alone. It also evaluates usability factors such as navigation clarity, form behavior, readability of results, responsiveness of interfaces, and continuity of user flow. This makes manual testing one of the most important evaluation methods in the project.

Table 6.1 Manual Testing Modules in AgroAI

Module	Manual Testing Focus
Registration	Valid and invalid account creation
Login	Correct and incorrect authentication
Disease Detection	Valid image upload and diagnosis result
History	Retrieval of previous records
Calendar	Display of scan dates
Farm Boundary Map	Display of saved field boundaries
Disease Trends	Display of disease related trends
Profile Management	Update and retrieval of user details

6.1.1 System Testing

System testing evaluates AgroAI as a complete integrated platform rather than as isolated modules. The purpose of system testing is to verify that the whole platform performs correctly when all components operate together. Since AgroAI combines the frontend, backend, database, and AI inference layer, system testing checks whether these layers interact correctly under realistic usage conditions.

In AgroAI, system testing includes full end to end scenarios. For example, a tester creates or logs into an account, uploads a plant image, waits for prediction, views the result, checks whether the result is stored in history, and then verifies whether that record appears later in the calendar or history view. This kind of testing confirms that the full sequence of operations

behaves correctly as one connected system. It also helps identify issues that might not appear when individual modules are tested separately.

System testing is particularly important for AgroAI because the platform relies on multiple dependent services. A user action begins on the interface, moves through backend processing, may require interaction with the database and AI model, and finally returns output to the user. If any part of this chain fails, the system does not meet its purpose. Therefore, end to end verification is necessary to ensure platform stability and reliability.

A typical system testing scenario in AgroAI includes the following:

1. The user enters the system through login.
2. The user uploads a plant leaf image.
3. The image is processed through backend and AI services.
4. The prediction result is displayed.
5. The diagnosis record is stored.
6. The stored record appears in history and calendar related views.

If all these steps succeed in a single flow, then the system demonstrates correct end to end behaviour. System testing therefore confirms that AgroAI functions as a complete software platform rather than as separate disconnected modules.

Table 6.1.1 System Testing of AgroAI

No.	Test Scenario	Expected Result	Result
1	Login and access dashboard	User enters main system successfully	Pass
2	Upload leaf image and receive diagnosis	Diagnosis is displayed correctly	Pass
3	Save diagnosis and view in history	Diagnosis appears in history	Pass
4	View scan date in calendar	Scan date is shown correctly	Pass
5	View saved farm boundary	Stored boundary appears on map	Pass

6.1.2 Unit Testing

Unit testing focuses on individual modules or components of the platform. Instead of testing the entire system as one flow, it checks whether each smaller function behaves correctly on its own. In AgroAI, unit testing is important because the platform contains several separate modules such as login validation, image validation, request handling, record retrieval, and profile update logic. By testing these modules independently, the development process becomes more controlled, and errors can be identified more precisely.

For example, the authentication module can be unit tested by checking whether valid input passes validation and whether invalid credentials are rejected properly. The image validation module can be tested by submitting supported and unsupported file formats to confirm that the system accepts only correct image types. The history retrieval module can be tested by checking whether the correct set of records is returned for a given authenticated user. The profile update module can also be tested separately by changing fields and verifying whether updates are processed correctly.

Unit testing is useful because it focuses on correctness at the smallest meaningful level. If each individual module performs correctly, the chances of stable overall system behaviour become much higher. In AgroAI, unit testing also supports maintainability, because developers can verify individual components even when other modules are still under refinement.

Typical unit tested areas in AgroAI include:

- input validation for registration and login
- image validation before disease detection
- backend request handling
- database save operation for diagnosis records
- record retrieval for history view
- update logic for profile management

This level of testing helps ensure that the internal logic of AgroAI remains dependable and easier to debug.

Table 6.1.2 Unit Testing of AgroAI

No.	Unit / Module	Test Objective	Expected Result	Result
1	Registration Validation	Check valid and invalid registration input	Accept valid, reject invalid	Pass
2	Login Validation	Verify credential checking	Correct login behaviour	Pass
3	Image Validation	Check supported image upload	Only valid images accepted	Pass
4	Diagnosis Save Logic	Check result storage	Diagnosis stored correctly	Pass
5	History Retrieval	Check user based record fetch	Correct records returned	Pass
6	Profile Update	Check profile modification logic	Updated data saved	Pass

6.1.3 Functional Testing

Functional testing verifies whether the features of AgroAI behave according to the defined requirements. While unit testing focuses on individual components and system testing focuses on the entire integrated flow, functional testing checks whether each system feature fulfills its intended purpose from the user's perspective. In other words, it confirms that the platform actually performs the services it is supposed to provide.

In AgroAI, functional testing covers major features such as registration, login, image upload, diagnosis result generation, history retrieval, calendar based scan display, farm boundary map visualization, disease trend viewing, and profile management. Each feature is tested by comparing expected behaviour with actual output. For example, if the user uploads a valid plant image, the system should provide a diagnosis result. If the user opens the history page, the system should show previously stored diagnosis records. If the user opens the map page, the saved field boundaries should appear correctly.

Functional testing is especially important in AgroAI because the system provides multiple user facing modules beyond the main AI diagnosis feature. It is not enough for the prediction workflow alone to work correctly; all surrounding features must also operate according to the

project requirements. This ensures that the platform delivers a complete and reliable experience.

Examples of functional testing in AgroAI include:

- registration creates a new account
- login grants access to protected features
- valid leaf image upload initiates disease prediction
- diagnosis result is shown after successful inference
- stored records are visible in history
- dates with scans are visible in the calendar
- saved farm boundaries are shown on the map
- disease trends page displays monitoring information
- profile changes are saved and reflected in the user interface

Functional testing therefore acts as direct evidence that AgroAI fulfills its intended system requirements in practice.

Table 6.1.3 Functional Testing of AgroAI

No.	Function	Test Objective	Expected Result	Result
1	Register Account	Create new user account	Account created successfully	Pass
2	Login	Access system with correct credentials	User enters dashboard	Pass
3	Upload Leaf Image	Start disease analysis	Image accepted and processed	Pass
4	View Diagnosis Result	Display prediction output	Result shown correctly	Pass
5	View History	Access previous records	History shown correctly	Pass
6	View Calendar	Show dates containing scans	Scan dates displayed	Pass

7	View Farm Boundaries	Display saved map boundaries	Boundaries shown correctly	Pass
8	View Disease Trends	Display trend information	Trends visible correctly	Pass
9	Manage Profile	Update account information	Changes saved successfully	Pass

6.1.4 Integration Testing

Integration testing checks whether the different modules of AgroAI communicate with each other correctly. Since the system is built from interconnected parts rather than a single isolated unit, this form of testing is necessary to verify that the interfaces between modules work properly. In AgroAI, the most important integrations are between the frontend and backend, the backend and database, and the backend and AI inference layer.

The **frontend backend integration** verifies that user actions performed on the interface correctly trigger backend operations. For example, when the user uploads a plant image, the frontend should successfully submit that image to the backend. When the user opens the history section, the frontend should request the correct diagnosis records from the backend. If this communication fails, the user facing platform becomes non functional even if the backend logic itself is correct.

The **backend database integration** verifies that the backend stores and retrieves data properly. In AgroAI, this includes saving new user accounts, saving diagnosis records, and retrieving history entries linked to a specific user. This integration is important because the system relies on persistent storage to maintain user specific information over time. If database interaction fails, history and personalized functionality are lost.

The **backend AI inference integration** is one of the most critical connections in AgroAI. This verifies that the backend successfully forwards uploaded plant images to the deployed inference layer and receives valid prediction responses. Since the AI model is deployed separately through Hugging Face Spaces, the backend must send and receive inference data correctly for the disease detection workflow to succeed. If this integration fails, the main purpose of AgroAI is not achieved.

Integration testing therefore ensures that the following major combinations work correctly:

- Frontend + Backend
- Backend + MongoDB
- Backend + Hugging Face AI inference service

Because AgroAI depends heavily on the coordination of these modules, integration testing is one of the most important parts of system evaluation.

Table 6.1.4 Integration Testing of AgroAI

No.	Integrated Modules	Test Objective	Expected Result	Result
1	Frontend + Backend	Verify user request handling	Requests processed correctly	Pass
2	Backend + Database	Verify data storage and retrieval	Records stored and fetched correctly	Pass
3	Backend + AI Service	Verify prediction request and response	AI result received successfully	Pass
4	Frontend + Backend + Database	Verify history retrieval flow	History shown correctly	Pass
5	Frontend + Backend + AI + Database	Verify complete diagnosis workflow	End to end diagnosis succeeds	Pass

6.2 Automated Testing

Automated testing refers to testing activities performed through scripts, tools, or framework supported validation instead of only through manual interaction. In AgroAI, automated testing supports consistency and repeatability in validation. While manual testing is essential for user experience and full feature observation, automated testing helps verify logic and repeated operations more efficiently.

In a project such as AgroAI, automated testing is most useful for predictable and repeatable tasks such as input validation, route testing, response verification, and module level checks. For example, backend endpoints for authentication or record retrieval can be tested repeatedly to verify whether they return the expected results. Similarly, frontend component behavior may be tested to confirm that form validation and interaction logic remain correct.

Automated testing is valuable because it reduces repeated manual effort and helps identify regressions when changes are introduced during development. If a module works correctly once but later changes cause unexpected behavior, automated tests help reveal such issues quickly. In AgroAI, this is especially relevant because the system contains interconnected modules and changes in one part may affect another.

If the automated testing coverage in AgroAI remains limited, the report may still describe it honestly by stating that the primary focus is on manual, functional, and integration testing, while automated testing is applied where practical for route validation, repeated logic checks, or form related verification. This keeps the chapter realistic and academically appropriate.

A concise description of automated testing in AgroAI may therefore include:

- validation of authentication related routes
- repeated verification of backend request response behaviour
- checking data retrieval routes
- basic frontend interaction validation where applicable

Table 6.2.1 Automated Testing of AgroAI

No.	Area	Automated Testing Purpose	Outcome
1	Authentication Routes	Verify repeated request response correctness	Verified
2	Diagnosis APIs	Check request handling and output structure	Verified
3	History Retrieval	Validate record fetch response	Verified
4	Form Validation	Check repeated input validation logic	Verified

6.3 AI Model Training Results and Evaluation

The AgroNet model was trained on the PlantVillage dataset using Kaggle's GPU environment (2× Tesla T4). The dataset contained 43,444 training images and 10,861 validation images across 38 plant disease classes. Training was conducted for 10 epochs using the Adam optimizer with a learning rate of 0.0001 and categorical cross-entropy as the loss function. Batch size was set to 32, and all images were resized to 128×128 pixels and normalized before being passed into the model.

Table 6.3.1 AgroNet Training Configuration

Parameter	Value
Training Images	43,444
Validation Images	10,861
Classes	38
Input Size	128×128×3
Epochs	10
Batch Size	32
Optimizer	Adam
Learning Rate	0.0001
Loss Function	Categorical Cross-Entropy
Hardware	2× NVIDIA Tesla T4 GPU (Kaggle)
Total Parameters	7,842,762

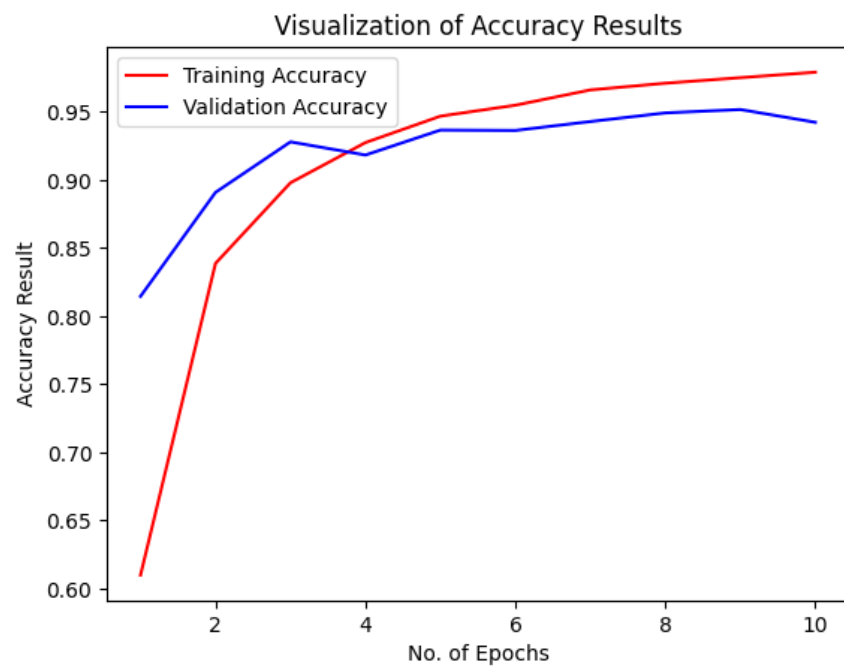


Figure 6-1 AgroNet Training and Validation Accuracy

The model achieves a final training accuracy of 97.82% and a validation accuracy of 94.24%. The consistent improvement across epochs confirms that AgroNet learns effectively from the PlantVillage dataset. The best validation accuracy of 95.18% was reached at Epoch 9, after which a slight decrease at Epoch 10 suggests the model is approaching its learning limit within 10 epochs without further scheduling. The gap between training and validation accuracy is approximately 3.6 percentage points, indicating that the model generalizes well without significant overfitting. The Dropout layers at rates 0.25 and 0.4 contribute to this regularization. These results confirm that AgroNet is suitable for deployment as the disease classification component of the AgroAI platform.

7 Conclusion and Future Work

7.1 Conclusion

AgroAI presents a cross platform AI powered software solution for plant disease diagnosis and agricultural support. The project is developed with the objective of providing users with a practical system through which plant leaf images can be analyzed and disease predictions can be generated in an accessible and structured way. By combining mobile and web access with backend services, database support, and an intelligent inference layer, AgroAI functions as more than a simple prediction tool; it operates as a complete digital platform for plant disease diagnosis, record maintenance, and user based interaction.

The project addresses an important problem in the agricultural domain: the timely and accurate identification of plant diseases. Traditional diagnosis methods often depend on manual observation or external expert support, which may be slow, inconsistent, or inaccessible in many situations. AgroAI responds to this challenge by introducing a software based environment in which users can upload plant images, receive disease predictions, and access stored diagnosis records through structured interfaces. In this way, the project demonstrates how modern computing technologies can be applied to solve a real world agricultural problem.

From a technical perspective, AgroAI integrates multiple layers of system design and implementation into a unified platform. The mobile application is developed using Flutter, while the web side is implemented using the MERN stack with TypeScript. The backend is developed using Node.js and Express.js, the database is maintained in MongoDB, and the plant

disease detection function is powered by a trained .h5 model deployed on Hugging Face Spaces for live inference. The successful coordination of these technologies shows that AgroAI is implemented as a complete full stack software project rather than a limited prototype.

The platform also includes features that extend its usefulness beyond direct disease prediction. In addition to the main diagnosis workflow, AgroAI provides user authentication, diagnosis history, a calendar view for scan dates, a farm boundary map view, disease trends monitoring, and profile management. These features contribute to the practicality of the platform by supporting ongoing interaction, organized record viewing, and visual monitoring of related information. As a result, the project demonstrates a broader understanding of software development in which the main intelligent function is supported by a complete user centered environment.

The design and implementation of AgroAI also reflect important academic concepts from software engineering, database systems, artificial intelligence, web engineering, mobile application development, and software project management. Through requirement analysis, architectural planning, modular implementation, and structured testing, the project transforms theoretical knowledge into a working system. This makes AgroAI not only a technically meaningful solution but also an academically valuable final year project.

The testing and evaluation of AgroAI show that the major system functions operate according to the intended implementation. The platform supports registration, login, image upload, AI based prediction, result display, record storage, history retrieval, and interface based access to calendar, map, and monitoring features. The testing chapter confirms that the frontend, backend, database, and AI inference service communicate correctly and support end to end operation. Therefore, the project successfully meets its main objective of providing a structured and accessible plant disease diagnosis platform.

If we had to identify one thing that made this project more challenging than expected, it was not the AI component, the model training itself was a well documented process. The harder part was building the surrounding system: making the backend communicate reliably with a remotely hosted inference service, keeping the mobile and web interfaces consistent while using different development frameworks, and ensuring that user records were correctly stored, linked, and retrieved across all features. AgroAI works not because any single component is particularly novel, but because all the components work together. Getting that integration right

is where most of the real engineering effort went, and it is what we are most satisfied with as a result.

7.2 Future Work

Although AgroAI is implemented as a functional and structured plant disease diagnosis platform, there are still several areas where the system can be improved and expanded in the future. These possible enhancements can increase the accuracy, usability, practical value, and overall impact of the platform.

One important area of future work is the improvement of the disease detection model. The effectiveness of plant disease prediction depends heavily on the quality of training data and the performance of the underlying model. In future versions, the model can be trained on larger and more diverse datasets so that it supports more plant species and a wider range of disease classes. Better training data can also improve prediction accuracy and make the system more reliable when used with real world agricultural images taken under different lighting conditions, backgrounds, and camera qualities.

The addition of treatment recommendations and plant care guidance represents an important future improvement. At present, AgroAI mainly focuses on disease diagnosis and record management. In the future, the platform can be extended so that after identifying a disease, it also provides treatment guidance, medicine suggestions, preventive measures, or fertilizer and care recommendations. This would make the system more practically useful because users would not only know which disease is present but would also receive guidance about the next possible actions.

Future work may also include multilingual support. Since agricultural users may belong to different linguistic backgrounds, adding multiple language options can make the platform more accessible and inclusive. For example, Urdu and regional language support can help local farmers understand diagnosis results, treatment guidance, and system instructions more comfortably. A multilingual interface would improve usability and allow a wider group of users to benefit from the platform.

The inclusion of offline or low connectivity support would also serve as a useful enhancement. Since the current architecture depends on live backend and inference services, an internet connection is required for normal system operation. In future versions, lightweight offline

support or cached interaction features can be explored so that users in low connectivity environments can still access certain platform functions. For example, the mobile application may temporarily store scan images or records and synchronize them with the server when internet access becomes available.

The farm boundary map module can also be expanded in future versions. At present, it supports saved field boundary viewing and map based representation. Future improvements may include richer location based features such as detailed field notes, crop specific boundary tagging, scan location marking, and integration with environmental or monitoring data. These additions can strengthen the agricultural management side of AgroAI and make the map feature more useful for field level crop health monitoring.

The disease trends module may also be enhanced in future work. Additional data sources, richer visualizations, and more advanced trend analysis can improve the usefulness of this feature. Instead of showing only basic monitoring information, future versions can provide deeper analytical insights such as disease frequency over time, repeated disease patterns, seasonal disease behavior, and crop wise disease comparison. This would help users better understand how disease activity changes across different periods and farm areas.

The expansion of the calendar and history features is also a possible improvement. In future versions, users may be able to filter records more easily, compare diagnosis results from different dates, and review scan activity through improved visual analytics. This would make the record management side of AgroAI stronger and more informative. A more advanced history module can also help users track treatment progress and compare plant health before and after taking corrective actions.

From a system perspective, future work can also include role based expansion. For example, a more complete administrative or expert oriented role may be added to provide monitoring, review, or feedback functions beyond normal user interaction. Agricultural experts may be allowed to review uncertain cases, verify low confidence predictions, or provide professional recommendations. This would allow AgroAI to evolve from a user centered diagnosis platform into a more collaborative agricultural support environment.

Finally, future work may focus on overall scalability and deployment refinement. As the number of users and diagnosis records increases, the system can be improved for broader adoption by strengthening deployment, performance handling, database optimization, and

service scalability. This is especially important if AgroAI is to be used beyond academic demonstration in practical real world settings.

Among these future directions, treatment and care recommendation is one of the most impactful enhancements. Identifying a disease is useful, but the value of the system increases significantly when the user also receives guidance about what to do next. Currently, AgroAI focuses on diagnosis, but adding a recommendation layer can transform it into a more complete agricultural decision support platform. This feature can include suggested treatment steps, preventive measures, links to verified agricultural guidance, and general care instructions. Such an enhancement would directly benefit farmers and smallholder growers by helping them move from disease identification toward practical crop protection actions.

References

- [1] Mohanty, S. P., Hughes, D. P., & Salathé, M. (2016). *Using deep learning for image based plant disease detection*. *Frontiers in Plant Science*, 7, 1419.
- [2] Ferentinos, K. P. (2018). *Deep learning models for plant disease detection and diagnosis*. *Computers and Electronics in Agriculture*, 145, 311–318.
- [3] Barbedo, J. G. A. (2016). *A review on the main challenges in automatic plant disease identification based on visible range images*. *Biosystems Engineering*, 144, 52–60.
- [4] Hughes, D. P., & Salathé, M. (2015). *An open access repository of images on plant health to enable the development of mobile disease diagnostics*. arXiv preprint arXiv:1511.08060.
- [5] Singh, D., Jain, N., Jain, P., Kayal, P., Kumawat, S., & Batra, N. (2019). *PlantDoc: A dataset for visual plant disease detection*. arXiv preprint arXiv:1911.10317.
- [6] Devarajan, D., et al. (2026). *AI based real time disease diagnosis in plants using deep learning*. *Scientific Reports*.
- [7] Saleem, M. H., Khanchi, S., Potgieter, J., & Arif, K. M. (2020). *Image Based Plant Disease Identification by Deep Learning Meta Architectures*. *Plants*, 9(11), 1451.
- [8] Wang, G., Sun, Y., & Wang, J. (2017). *Automatic image based plant disease severity estimation using deep learning*. *Computational Intelligence and Neuroscience*, 2017, 2917536.
- [9] Khan, R. U., Khan, K., Albattah, W., & Qamar, A. M. (2021). *Image Based Detection of Plant Diseases: From Classical Machine Learning to Deep Learning Journey*. *Wireless Communications and Mobile Computing*, 2021, 5541859.
- [10] Barbedo, J. G. A., Koenigkan, L. V., Halfeld Vieira, B. A., Costa, R. V., Nechet, K. L., et al. (2018). *Annotated Plant Pathology Databases for Image Based Detection and Recognition of Diseases*. *IEEE Latin America Transactions*, 16, 1749–1757.
- [11] Mittal, U., & others. (2023). *Plant Disease Detection and Classification*. Review article indexed in PubMed Central.
- [12] Demilie, W. B. (2024). *Plant disease detection and classification techniques: a comparative study of the performances*. *Journal of Big Data*, 11(1).

- [13] Paçal, I., et al. (2024). *A systematic review of deep learning techniques for plant disease detection*. Knowledge Based Systems, 274, 109444.
- [14] Upadhyay, A., Chandel, N. S., Singh, K. P., Chakraborty, S. K., Nandede, B. M., Kumar, M., Subeesh, A., Upendar, K., Salem, A., & Elbeltagi, A. (2025). *Deep learning and computer vision in plant disease detection: a comprehensive review of techniques, models, and trends in precision agriculture*. Artificial Intelligence Review, 58, 92.
- [15] Krishna, M. S., et al. (2025). *Plant Leaf Disease Detection Using Deep Learning: A Multi Condition Approach*. AI, 8(1), 4.
- [16] Ashurov, A. Y., et al. (2025). *Enhancing plant disease detection through deep learning*. Frontiers in Plant Science.
- [17] Aldakheel, E. A., et al. (2024). *Detection and identification of plant leaf diseases using YOLOv4*. Article available in PubMed Central.
- [18] Ali, A. H., et al. (2024). *An ensemble of deep learning architectures for accurate plant disease classification*. Neurocomputing / related indexed journal article on ensemble DL for plant disease classification.
- [19] Leite, D., et al. (2024). *Advancements and outlooks in utilizing Convolutional Neural Networks for plant disease severity assessment: a systematic literature review*. Smart Agricultural Technology.
- [20] Sankhe, S. R., & Ambhaikar, A. (2025). *Plant disease detection and classification techniques: A review*. Journal of Information Technology Research / Sage indexed review source.
- [21] Bagga, M., et al. (2024). *Image based detection and classification of plant diseases under complex field conditions using deep learning*. Wiley online article on crop disease detection.
- [22] Salman, Z., et al. (2025). *Plant disease classification in the wild using vision models*. Article available in PubMed Central.
- [23] Khan, A. T., et al. (2023). *Plant disease detection model for edge computing devices*. Frontiers in Plant Science.
- [24] Das, S., et al. (2022). *Application of machine learning: a recent advancement in plant diseases detection*. Journal of Plant Protection Research.